VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB RECORD

Artificial Intelligence (23CS5PCAIN)

Submitted by

Jeevan A (1BM22CS119)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
Sep-2024 to Jan-2025

B.M.S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled " Bio Inspired Systems (23CS5BSBIS)" carried out by **Jeevan A(1BM22CS119),** who is bonafide student of **B.M.S. College of Engineering.** It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements of the above mentioned subject and the work prescribed for the said degree.

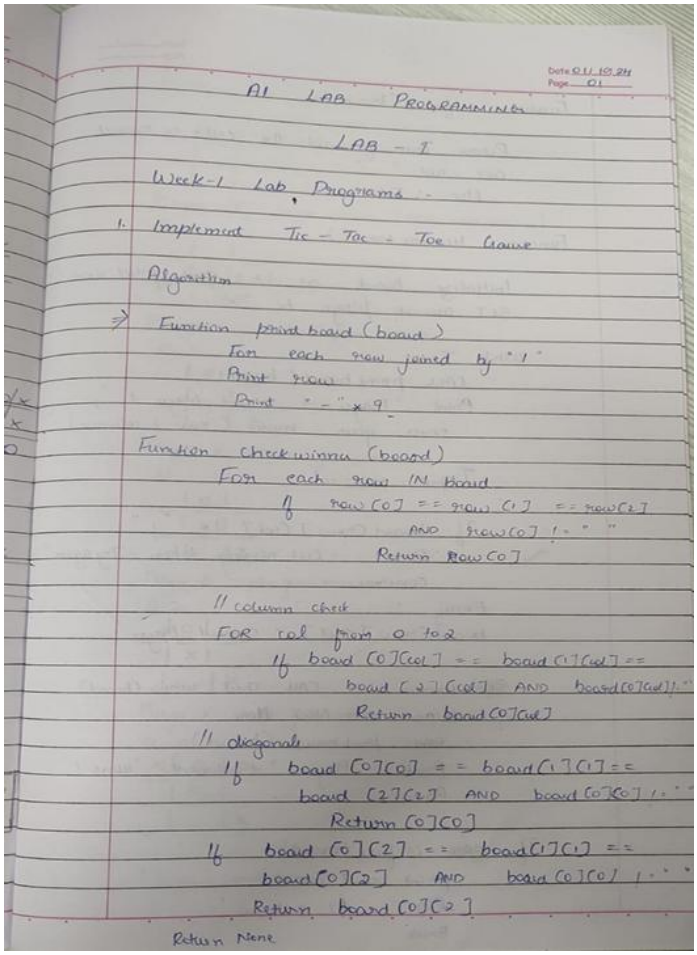
Dr. Pallavi G B Associate Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE

INDEX

Github Link:
Jeevan-017/AI-LAB

1. Implement Tic –Tac –Toe Game.

ALGORITHM



AI LAB PROGRAMMING

LAB - 1

Week-1 Lab Programs

1. Implement Tic - Tac - Toe Game

Algorithm

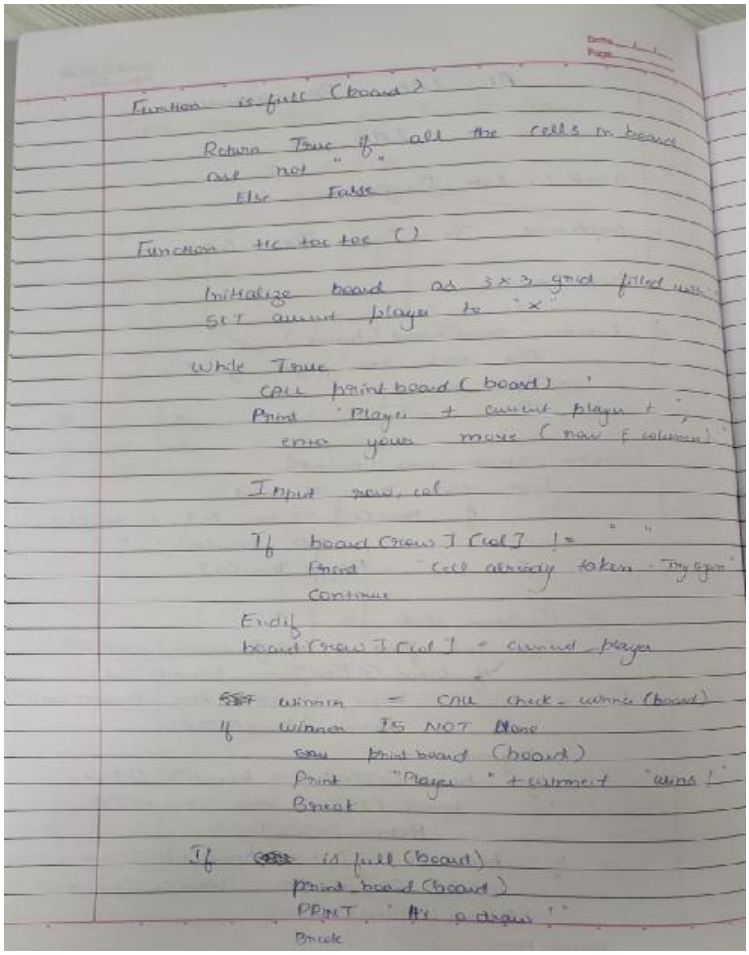
Function print board (board)
For each row joined by " | "
Print row
Print "-" x 9

Function check winner (board)
For each row IN board
If row[0] == row[1] == row[2]
AND row[0] != " "
Return Row[0]

// column check
FOR col from 0 to 2
If board[0][col] == board[1][col] ==
board[2][col] AND board[0][col] != " "
Return board[0][col]

// diagonals
If board[0][0] == board[1][1] ==
board[2][2] AND board[0][0] != " "
Return [0][0]
If board[0][2] == board[1][1] ==
board[0][2] AND board[0][0] != " "
Return board[0][2]

Return None



Function is full (board)

Return True if all the cells in board are not " "
Else False

Function tic tac toe ()

Initialize board as 3x3 grid filled with " "
SET current player to "x"

While True
CALL print board (board)
Print "Player " + current player + ", enter your move (row & column)"

Input row, col

If board[row][col] != " "
Print "Cell already taken. Try again"
Continue
Endif
board[row][col] = current player

winner = CALL check winner (board)
If winner IS NOT None
CALL print board (board)
Print "Player " + winner + " WINS!"
Break

If is full (board)
print board (board)
PRINT "It's a draw!"
Break

CODE:

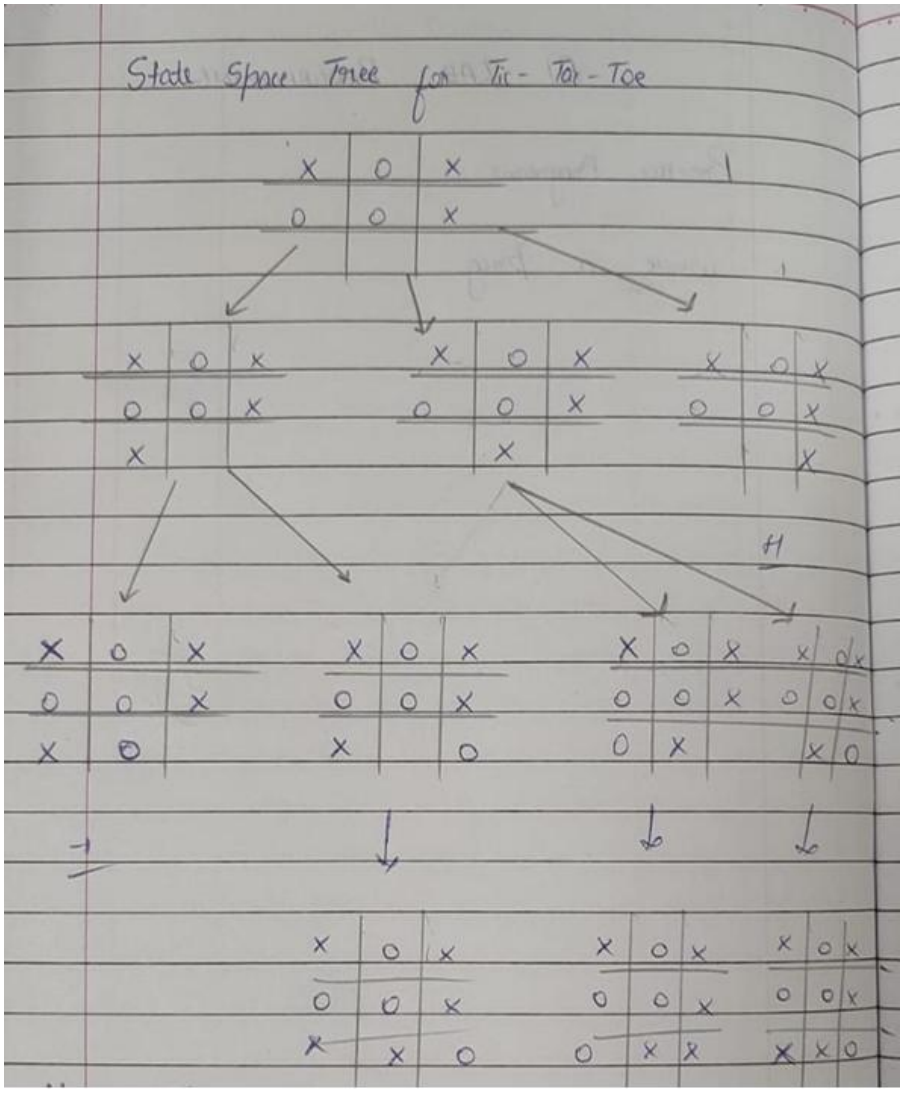
```
import random

board = ["-", "-", "-",
"-", "-", "-",
"-", "-", "-"]

def p_board(player):
print(board[0] + "|" + board[1] + "|" + board[2])
print(board[3] + "|" + board[4] + "|" + board[5])
print(board[6] + "|" + board[7] + "|" + board[8])

def check_win():

for i in range(0, 7, 3):
if board[i] == board[i + 1] == board[i + 2] and
board[i] != '-':
return board[i]

for i in range(3):
if board[i] == board[i + 3] == board[i + 6] and
board[i] != '-':
return board[i]

if board[0] == board[4] == board[8] and board[0] !=
'-':
return board[0]
if board[2] == board[4] == board[6] and board[2] !=
'-':
return board[2]
return None
def check_tie():
if '-' not in board:
return True
return False

def play_game():
current_player = "X"
game_over = False

while not game_over:
```

```
p_board(current_player)

if current_player == "X":
print("It's your turn (X).")
try:
position = int(input("Choose a position from 1-9:
")) - 1
if position < 0 or position > 8 or board[position]
!= '-':
print("Invalid move. Try again.")
continue
except ValueError:
print("Invalid input. Please enter a number
between 1 and 9.")
continue
else:
print("Computer's turn (O).")
available_moves = [i for i, spot in
enumerate(board) if spot == '-']
position = random.choice(available_moves)

board[position] = current_player
winner = check_win()

if winner:
p_board(current_player)
print(winner + " won!")
game_over = True
elif check_tie():
p_board(current_player)
print("It's a tie!")
game_over = True
else:
current_player = "O" if current_player == "X" else
"X"

play_game()
```

OUTPUT:

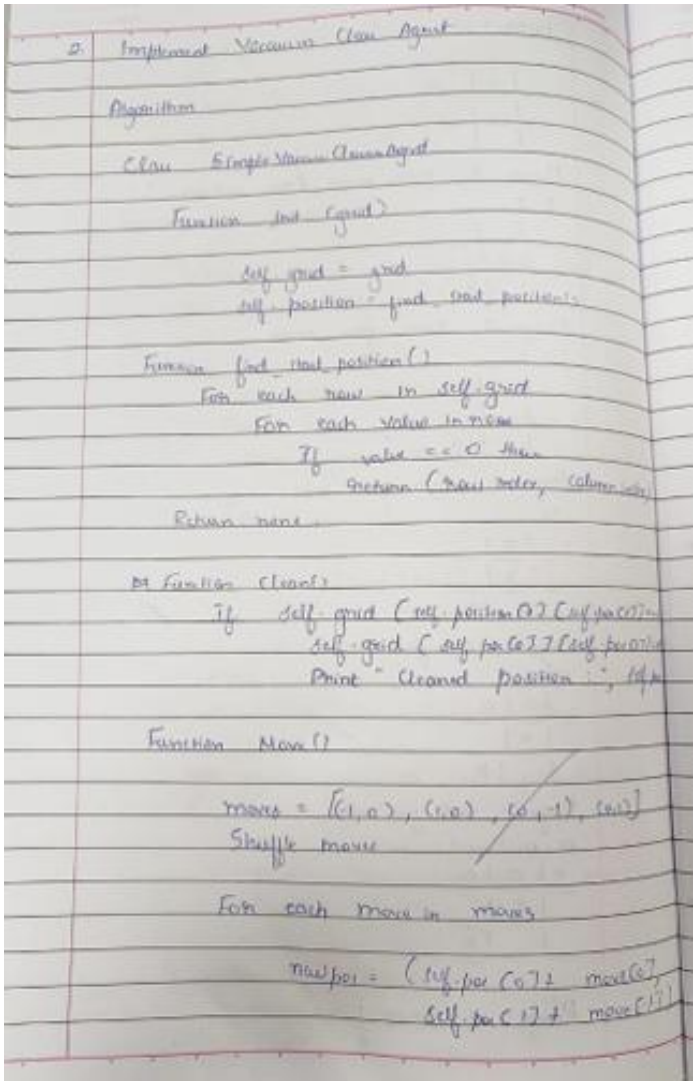
```
-|-|-
-|-|-
-|-|-
It's your turn (X).
Choose a position from 1-9: 5
-|-|-
-|X|-
-|-|-
Computer's turn (O).
-|O|-
-|X|-
-|-|-
It's your turn (X).
Choose a position from 1-9: 7
-|O|-
-|X|-
X|-|-
Computer's turn (O).
-|O|O
-|X|-
X|-|-
It's your turn (X).
Choose a position from 1-9: 1
X|O|O
-|X|-
X|-|-
Computer's turn (O).
X|O|O
-|X|O
X|-|-
It's your turn (X).
Choose a position from 1-9: 4
X|O|O
X|X|O
X|-|-
X won!
```

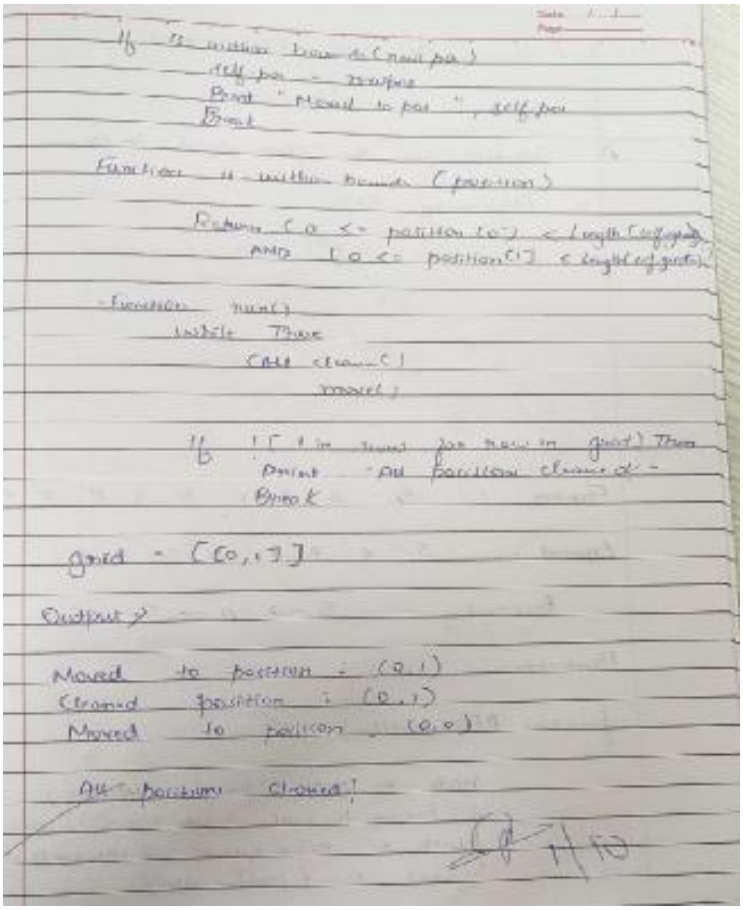
STATE-SPACE TREE:



2.Implement Vacuum Cleaner

ALGORITHM:





CODE:

```
def vacuum_world():
    goal_state = {'A': '0', 'B': '0'}
    cost = 0
    location_input = input("Enter Location of Vacuum: ")
    status_input = input("Enter status of " + location_input + " (0 for Clean, 1 for
Dirty): ")
    status_input_complement = input("Enter status of other room: ")

    print("Initial Location Condition: " + str(goal_state))

    if location_input == 'A':

        print("Vacuum is placed in Location A")

        if status_input == '1':
            print("Location A is Dirty.")

            goal_state['A'] = '0'
            cost += 1
            print("Cost for CLEANING A: " + str(cost))
            print("Location A has been Cleaned.")

            if status_input_complement == '1':
                print("Location B is Dirty.")
                print("Moving right to the Location B.")
                cost += 1
                print("COST for moving RIGHT: " + str(cost))

                goal_state['B'] = '0'
                cost += 1
                print("COST for SUCK: " + str(cost))
                print("Location B has been Cleaned.")
            else:
                print("No action. " + str(cost))
                print("Location B is already clean.")

        if status_input == '0':
```

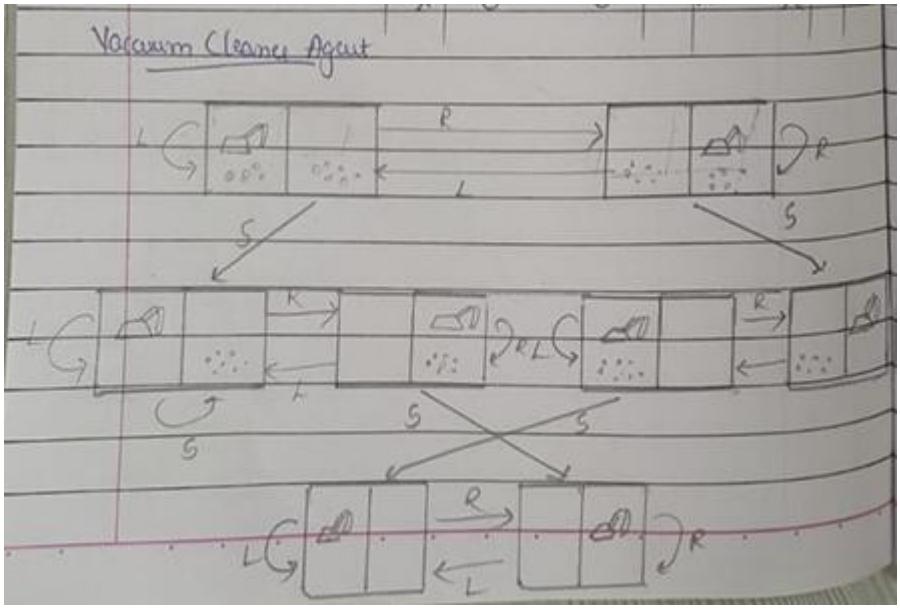
```
        print("Location A is already clean.")

        if status_input_complement == '1':
            print("Location B is Dirty.")
            print("Moving RIGHT to the Location B.")
            cost += 1
            print("COST for moving RIGHT: " + str(cost))

            goal_state['B'] = '0'
            cost += 1
            print("Cost for SUCK: " + str(cost))
            print("Location B has been Cleaned.")
        else:
            print("No action. " + str(cost))
            print("Location B is already clean.")

else:
    print("Vacuum is placed in Location B")


    if status_input == '1':
        print("Location B is Dirty.")

        goal_state['B'] = '0'
        cost += 1
        print("COST for CLEANING: " + str(cost))
        print("Location B has been Cleaned.")

        if status_input_complement == '1':
            print("Location A is Dirty.")
            print("Moving LEFT to the Location A.")
            cost += 1
            print("COST for moving LEFT: " + str(cost))

            goal_state['A'] = '0'
            cost += 1
            print("COST for SUCK: " + str(cost))
            print("Location A has been Cleaned.")
        else:
```

```
            print("Location A is already clean.")
        if status_input_complement == '1':
            print("Location A is Dirty.")
            print("Moving LEFT to the Location A.")
            cost += 1
            print("COST for moving LEFT: " + str(cost))

            goal_state['A'] = '0'
            cost += 1
            print("Cost for SUCK: " + str(cost))
            print("Location A has been Cleaned.")
        else:
            print("No action. " + str(cost))
            print("Location A is already clean.")


    print("GOAL STATE: ")
    print(goal_state)
    print("Performance Measurement: " + str(cost))


vacuum_world()
```

OUTPUT:

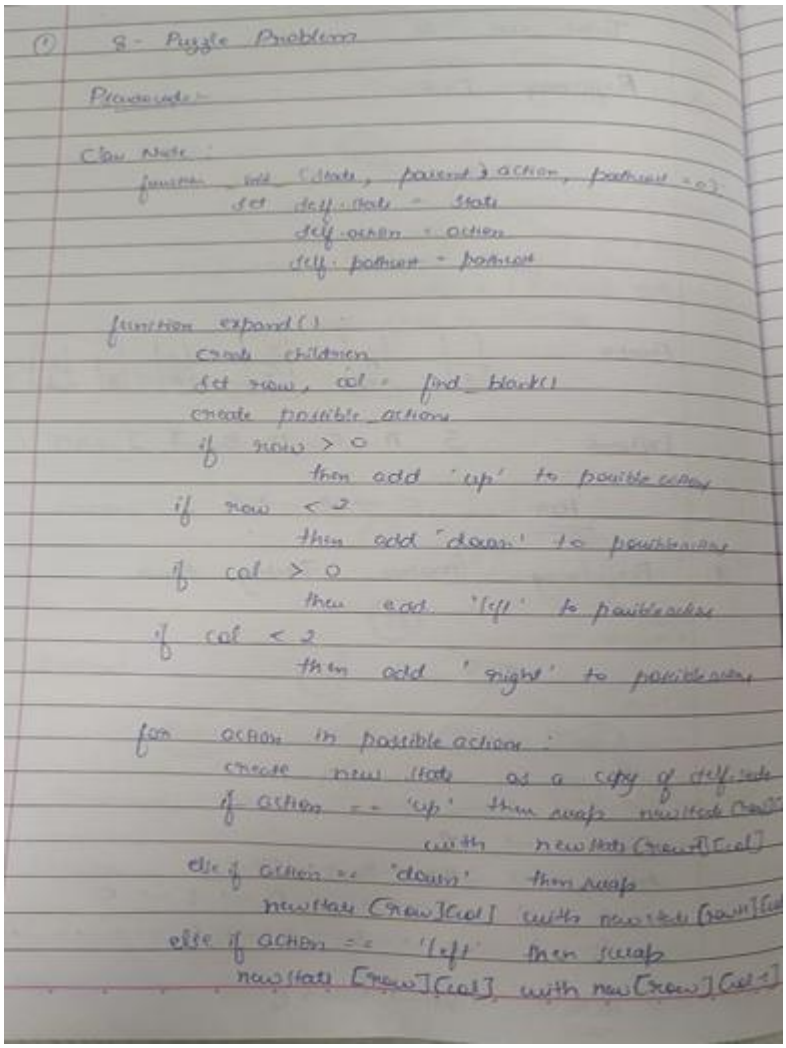
```
Enter the initial location of the vacuum cleaner (A or B): A
Enter the status of room A (0 for Clean, 1 for Dirty): 1
Enter the status of the other room (B) (0 for Clean, 1 for Dirty): 1

Initial Location Condition: {'A': '1', 'B': '1'}

Vacuum cleaner is placed in room A.
Room A is Dirty. Cleaning room A...
Cost for cleaning room A: 1

Room B is also Dirty. Moving to room B...
Cost for moving to room B: 2
Cleaning room B...
Cost for cleaning room B: 3

GOAL STATE: {'A': '0', 'B': '0'}
Total cost for cleaning: 3
```

STATE-SPACE TREE:



3.Solve 8 puzzle problems using DFS

ALGORITHM:



① 8- Puzzle Problem

Procedure:-

Class Node:
function init (state, parent, action, pathcost = 0)
set self.state = state
self.action = action
self.pathcost = pathcost

function expand()
create children
Set row, col = find_blank()
create possible_actions
if row > 0
then add 'up' to possible actions
if row < 2
then add 'down' to possible actions
if col > 0
then add 'left' to possible actions
if col < 2
then add 'right' to possible actions

for action in possible actions:
create new state as a copy of self.state
if action == 'up' then swap newState[row][col]
with newState[row-1][col]
else if action == 'down' then swap
newState[row][col] with newState[row+1][col]
else if action == 'left' then swap
newState[row][col] with new[row][col-1]

CODE:

```
class Node:
    def __init__(self, puzzle, x, y, parent=None):
        self.puzzle = [row[:] for row in puzzle]
        self.x = x
        self.y = y
        self.parent = parent

goal = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 0]
]

dx = [-1, 1, 0, 0]
dy = [0, 0, -1, 1]

def is_goal(puzzle):
    return puzzle == goal

def print_puzzle(puzzle):
    for row in puzzle:
        print(' '.join(str(x) if x != 0 else ' ' for x in row))
    print()

def is_valid(x, y):
    return 0 <= x < 3 and 0 <= y < 3

def count_inversions(puzzle):
    flat_puzzle = [num for row in puzzle for num in row if num != 0]
    inversions = 0
    for i in range(len(flat_puzzle)):
        for j in range(i + 1, len(flat_puzzle)):
            if flat_puzzle[i] > flat_puzzle[j]:
                inversions += 1
    return inversions

def is_solvable(puzzle):
```

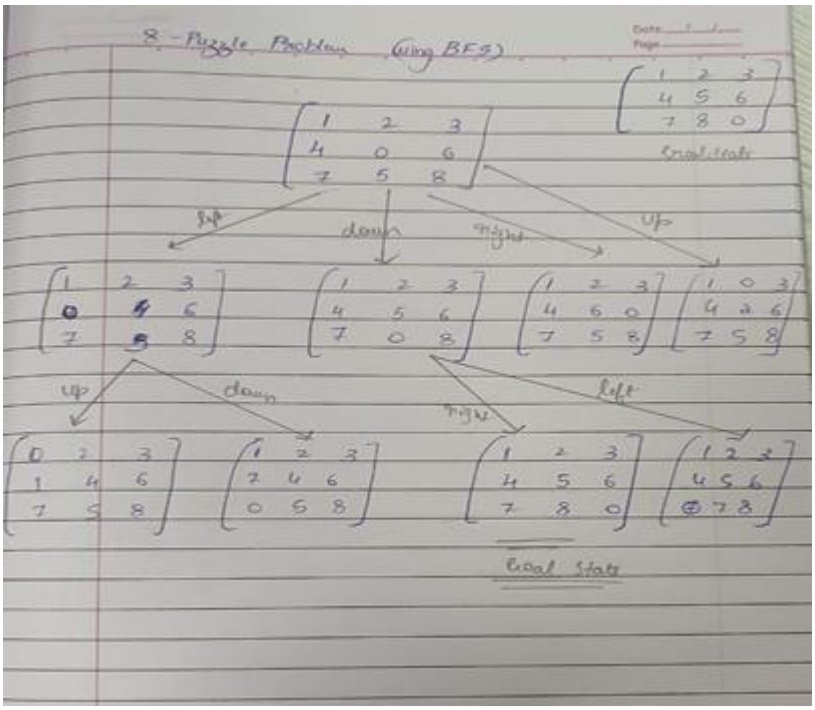
```
    return count_inversions(puzzle) % 2 == 0

def dfs(root):
    stack = [root]
    visited = set()

    while stack:
        node = stack.pop()

        if is_goal(node.puzzle):
            print("Solution found:")
            path = []
            while node:
                path.append(node.puzzle)
                node = node.parent
            for state in reversed(path):
                print_puzzle(state)
            return True

        puzzle_tuple = tuple(map(tuple, node.puzzle))
        if puzzle_tuple in visited:
            continue
        visited.add(puzzle_tuple)

        for i in range(4):
            new_x = node.x + dx[i]
            new_y = node.y + dy[i]

            if is_valid(new_x, new_y):
                new_puzzle = [row[:] for row in node.puzzle]
                new_puzzle[node.x][node.y], new_puzzle[new_x][new_y] =
new_puzzle[new_x][new_y], new_puzzle[node.x][node.y]

                new_puzzle_tuple = tuple(map(tuple, new_puzzle))
                if new_puzzle_tuple not in visited:
                    new_node = Node(new_puzzle, new_x, new_y, node)
                    stack.append(new_node)

    print("No solution found.")
```

```
    return False

def main():
    initial_puzzle = [
        [1, 2, 3],
        [4, 5, 6],
        [7, 0, 8]
    ]

    print("Initial Puzzle:")
    print_puzzle(initial_puzzle)

    if not is_solvable(initial_puzzle):
        print("The provided puzzle is unsolvable.")
        return

    try:
        x, y = [(i, j) for i in range(3) for j in range(3) if initial_puzzle[i][j] == 0][0]
    except IndexError:
        print("Invalid puzzle: No blank space (0) found.")
        return

    root = Node(initial_puzzle, x, y)

    if not dfs(root):
        print("Failed to find a solution.")

if __name__ == "__main__":
    main()
```

OUTPUT:

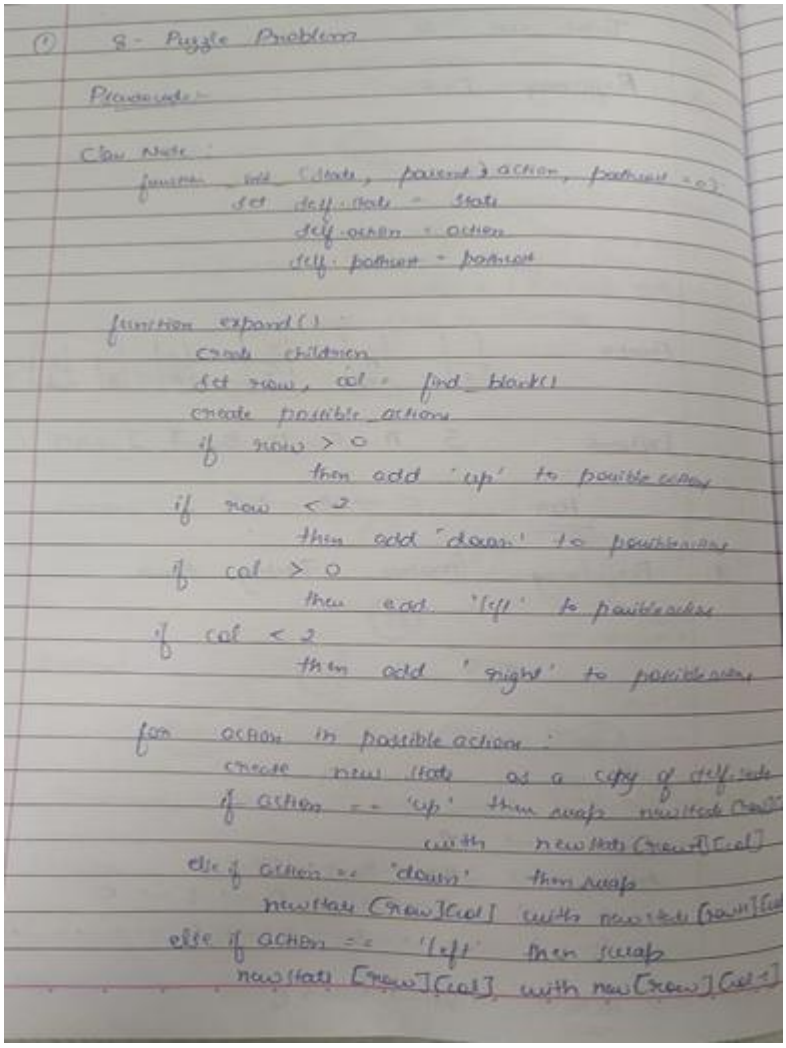
```
Initial Puzzle:
1 2 3
4 5 6
7   8

Solution found:
1 2 3
4 5 6
7   8

1 2 3
4 5 6
7 8
```

STATE-SPACE TREE:



4.Solve 8 puzzle problems using BFS

ALGORITHM:



① 8- Puzzle Problem

Pseudocode:

Class Node:
function __init__(state, parent, action, pathcost=0):
self.state = state
self.action = action
self.pathcost = pathcost

function expand():
create children
set row, col = find_blank()
create possible_actions
if row > 0
then add 'up' to possible actions
if row < 2
then add 'down' to possible actions
if col > 0
then add 'left' to possible actions
if col < 2
then add 'right' to possible actions

for action in possible actions:
create new state as a copy of self.state
if action == 'up' then swap newState[row][col]
with newState[row-1][col]
else if action == 'down' then swap
newState[row][col] with newState[row+1][col]
else if action == 'left' then swap
newState[row][col] with new[row][col-1]

CODE:

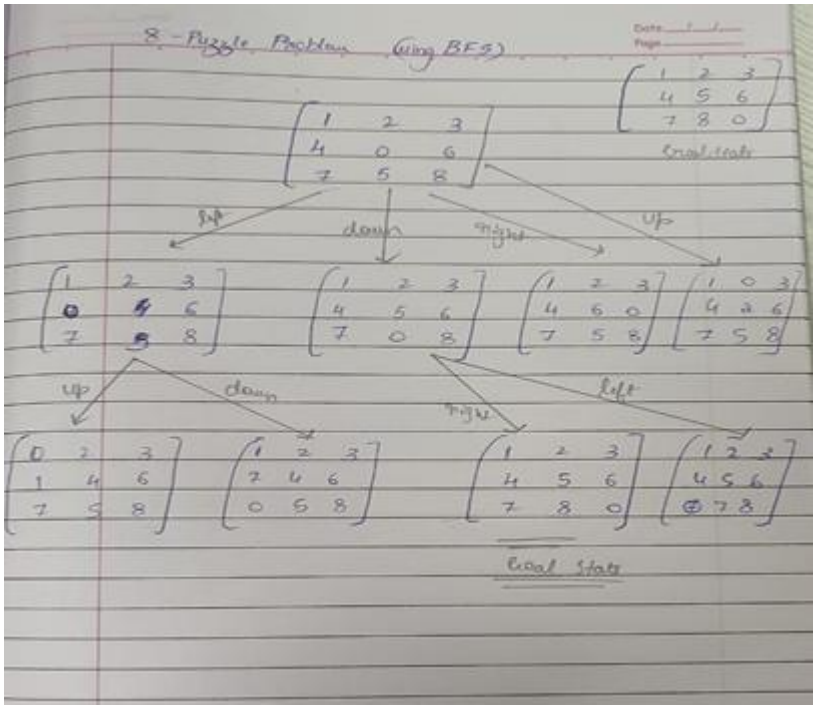
```
from collections import deque

class Node:
    def __init__(self, puzzle, x, y, parent=None):
        self.puzzle = [row[:] for row in puzzle]
        self.x = x
        self.y = y
        self.parent = parent
goal = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 0]
]
dx = [-1, 1, 0, 0]
dy = [0, 0, -1, 1]

def is_goal(puzzle):
    return puzzle == goal

def print_puzzle(puzzle):
    for row in puzzle:
        print(' '.join(str(x) if x != 0 else ' ' for x in row))
    print()

def is_valid(x, y):
    return 0 <= x < 3 and 0 <= y < 3

def bfs(root):
    queue = deque([root])
    visited = set()

    while queue:
        node = queue.popleft()

        if is_goal(node.puzzle):
            print("Solution found:")
            path = []
            while node:
                path.append(node.puzzle)
                node = node.parent
            for state in reversed(path):
```

```
            print_puzzle(state)
            return True

        puzzle_tuple = tuple(map(tuple, node.puzzle))
        if puzzle_tuple in visited:
            continue
        visited.add(puzzle_tuple)

        for i in range(4):
            new_x = node.x + dx[i]
            new_y = node.y + dy[i]

            if is_valid(new_x, new_y):
                new_puzzle = [row[:] for row in node.puzzle]
                new_puzzle[node.x][node.y], new_puzzle[new_x][new_y] =
new_puzzle[new_x][new_y], new_puzzle[node.x][node.y]

                new_puzzle_tuple = tuple(map(tuple, new_puzzle))
                if new_puzzle_tuple not in visited:
                    new_node = Node(new_puzzle, new_x, new_y, node)
                    queue.append(new_node)

    print("No solution found.")
    return False

def main():
    initial_puzzle = [
        [1, 2, 3],
        [4, 0, 5],
        [7, 8, 6]
    ]
    print("Initial Puzzle:")
    print_puzzle(initial_puzzle)

    x, y = [(i, j) for i in range(3) for j in range(3) if initial_puzzle[i][j] == 0][0]

    root = Node(initial_puzzle, x, y)

    if not bfs(root):
        print("Failed to find a solution.")

if __name__ == "__main__":
    main()
```

OUTPUT:

```
#Dhanush C  1BM22CS085
Initial Puzzle:
1 2 3
4   5
7 8 6

Solution found:
1 2 3
4   5
7 8 6

1 2 3
4 5
7 8 6

1 2 3
4 5 6
7 8
```

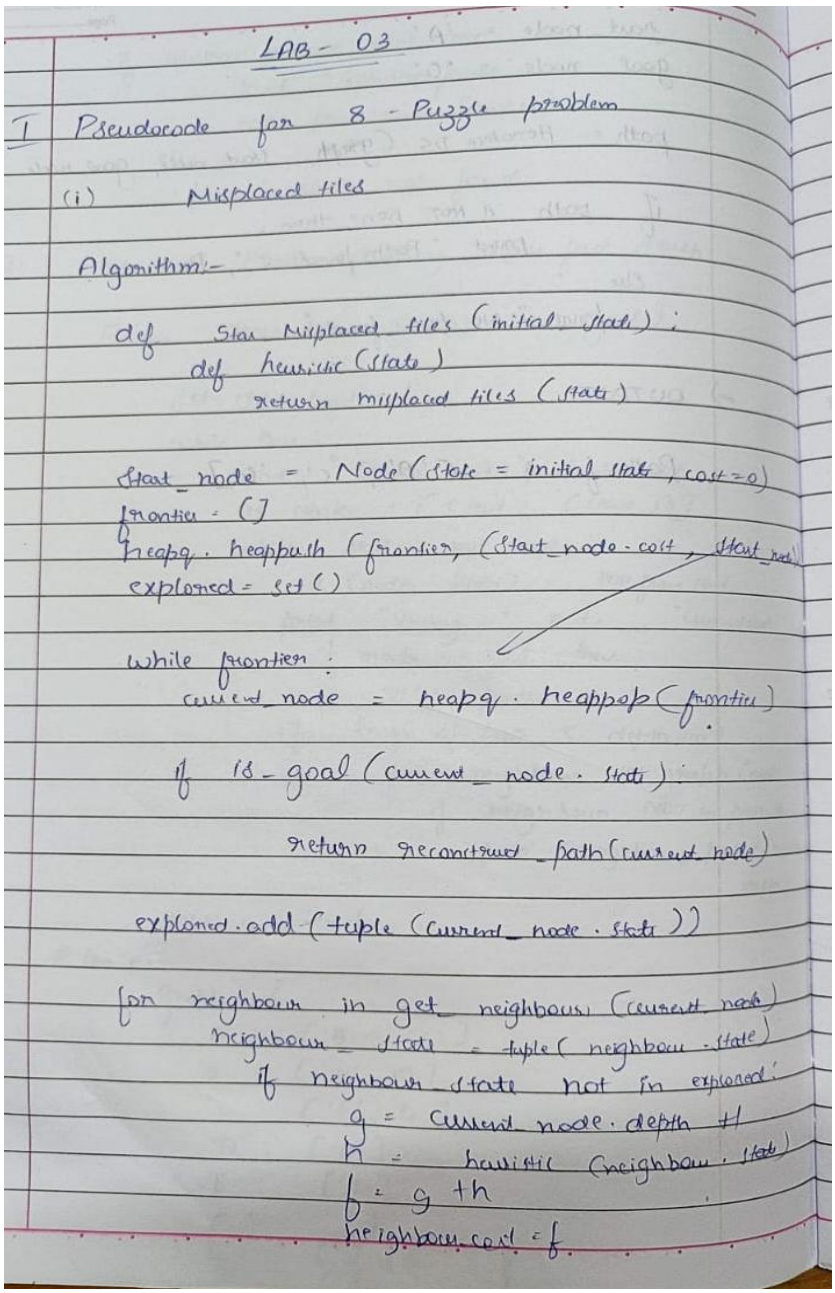
STATE-SPACE TREE:



5.Implement A* (A star) search algorithm

a.Misplacing tiles

ALGORITHM:



LAB - 03

I Pseudocode for 8 - Puzzle problem

(i) Misplaced tiles

Algorithm:-

```
def Star Misplaced tiles (initial_state):
    def heuristic (state)
        return misplaced tiles (state)

start_node = Node (state = initial state, cost=0)
frontier = []
heapq.heappush (frontier, (start_node.cost, start_node
explored = set()

while frontier:
    current_node = heapq.heappop(frontier)

    if is_goal (current_node.state):

        return reconstruct_path(current_node)

explored.add (tuple (current_node.state))

for neighbour in get_neighbours (current_node)
    neighbour_state = tuple (neighbour.state)
    if neighbour_state not in explored:
        g = current_node.depth +1
        h = heuristic (neighbour.state)
        f = g + h
        neighbour.cost = f.
```

CODE:

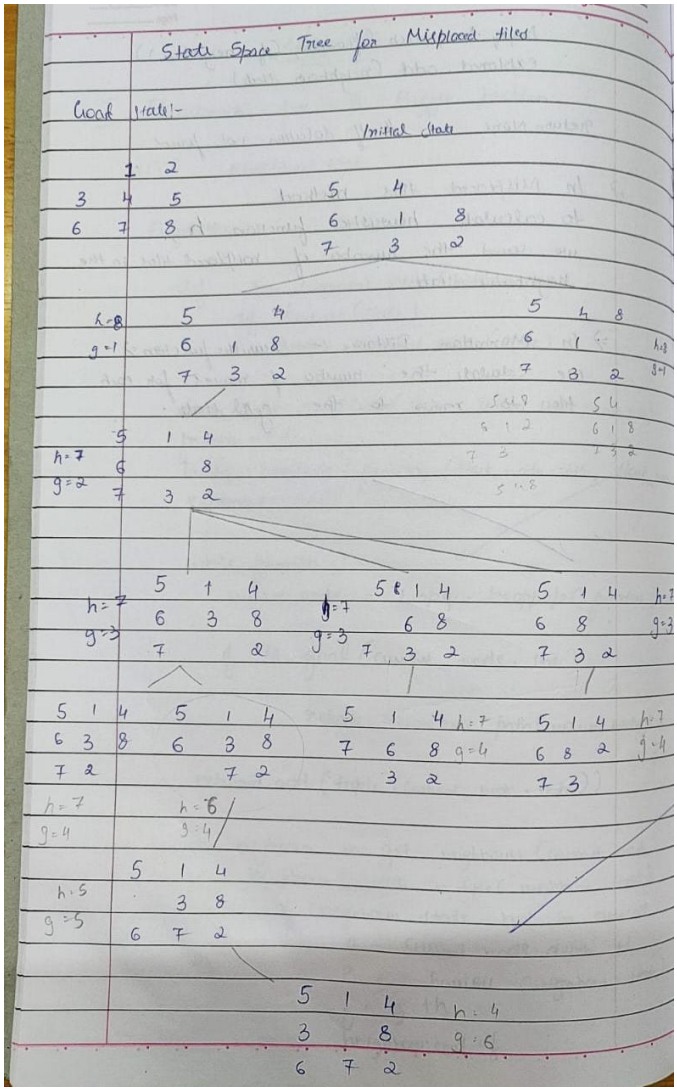
```
import heapq

def misplaced_tiles(state, goal):
    return sum(1 for i in range(3) for j in range(3)
               if state[i][j] != goal[i][j] and state[i][j] != 0)

def get_neighbors(state):
    neighbors = []
    blank = [(i, j) for i in range(3) for j in range(3) if state[i][j] == 0][0]
    possible_moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]
    x, y = blank

    for dx, dy in possible_moves:
        nx, ny = x + dx, y + dy
        if 0 <= nx < 3 and 0 <= ny < 3:
            new_state = [row[:] for row in state]
            new_state[x][y], new_state[nx][ny] = new_state[nx][ny], new_state[x][y]
            neighbors.append(new_state)

    return neighbors

def print_path(path):
    for state in path:
        for row in state:
            print(row)
        print()

def astar_misplaced(start, goal):
    open_list = []
```

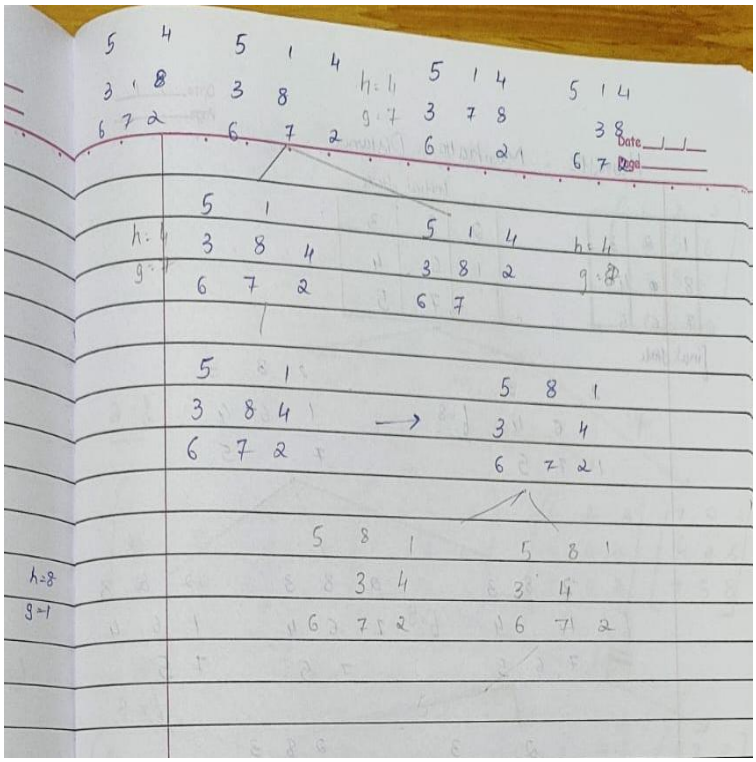
```
    heapq.heappush(open_list, (0 + misplaced_tiles(start, goal), start, 0, []))  # (f, state, g, path)
    visited = set()

    while open_list:
        f, current, g, path = heapq.heappop(open_list)
        path = path + [current]

        if current == goal:
            print("Solution Found (Misplaced Tiles):")
            print_path(path)
            return g  # Return the depth (number of moves)

        for neighbor in get_neighbors(current):
            neighbor_tuple = tuple(map(tuple, neighbor))
            if neighbor_tuple not in visited:
                visited.add(neighbor_tuple)
                h = misplaced_tiles(neighbor, goal)
                heapq.heappush(open_list, (g + 1 + h, neighbor, g + 1, path))

    return -1  # No solution found

# Example usage
start = [[5, 4, 0], [6, 1, 8], [7, 3, 2]]
goal = [[0, 1, 2], [3, 4, 5], [6, 7, 8]]
astar_misplaced(start, goal)
```

OUTPUT:

```
Solution Found (Misplaced Tiles):
[5, 4, 0]
[6, 1, 8]
[7, 3, 2]

[5, 0, 4]
[6, 1, 8]
[7, 3, 2]

[0, 5, 4]
[6, 1, 8]
[7, 3, 2]

[6, 5, 4]
[0, 1, 8]
[7, 3, 2]

[6, 5, 4]
[1, 0, 8]
[7, 3, 2]

[6, 5, 4]
[1, 3, 8]
[7, 0, 2]

[6, 5, 4]
[1, 3, 8]
[7, 2, 0]

[6, 5, 4]
[1, 3, 0]
[7, 2, 8]

[6, 5, 0]
[1, 3, 4]
[7, 2, 8]

[6, 0, 5]
[1, 3, 4]
[7, 2, 8]

[6, 3, 5]
[1, 0, 4]
[7, 2, 8]

[3, 1, 5]
[6, 0, 4]
[7, 2, 8]

[3, 1, 5]
[6, 2, 4]
[7, 0, 8]

[3, 1, 5]
[6, 2, 4]
[0, 7, 8]

[3, 1, 5]
[0, 2, 4]
[6, 7, 8]

[0, 1, 5]
[3, 2, 4]
[6, 7, 8]

[1, 0, 5]
[3, 2, 4]
[6, 7, 8]

[1, 2, 5]
[3, 0, 4]
[6, 7, 8]

[1, 2, 5]
[3, 4, 0]
[6, 7, 8]

[1, 2, 0]
[3, 4, 5]
[6, 7, 8]

[1, 0, 2]
[3, 4, 5]
[6, 7, 8]

[0, 1, 2]
[3, 4, 5]
[6, 7, 8]
```

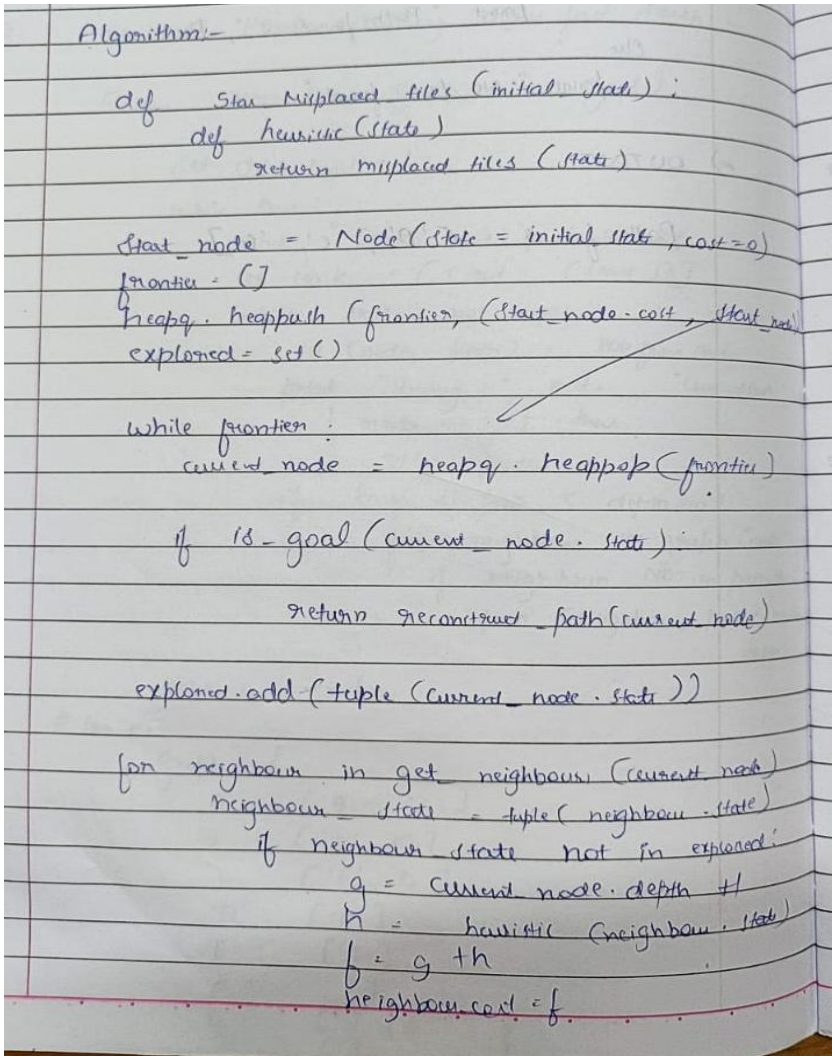
STATE-SPACE TREE:





b. Manhattan Distance

ALGORITHM:



CODE :

```
import heapq

def manhattan_distance(state, goal):
    distance = 0
```

```
    for i in range(3):
        for j in range(3):
            if state[i][j] != 0:
                goal_x, goal_y = [(x, y) for x in range(3) for y in range(3)
                                  if goal[x][y] == state[i][j]][0]
                distance += abs(i - goal_x) + abs(j - goal_y)
    return distance


def get_neighbors(state):
    neighbors = []
    blank = [(i, j) for i in range(3) for j in range(3) if state[i][j] == 0][0]
    possible_moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]
    x, y = blank

    for dx, dy in possible_moves:
        nx, ny = x + dx, y + dy
        if 0 <= nx < 3 and 0 <= ny < 3:
            new_state = [row[:] for row in state]
            new_state[x][y], new_state[nx][ny] = new_state[nx][ny], new_state[x][y]
            neighbors.append(new_state)
```

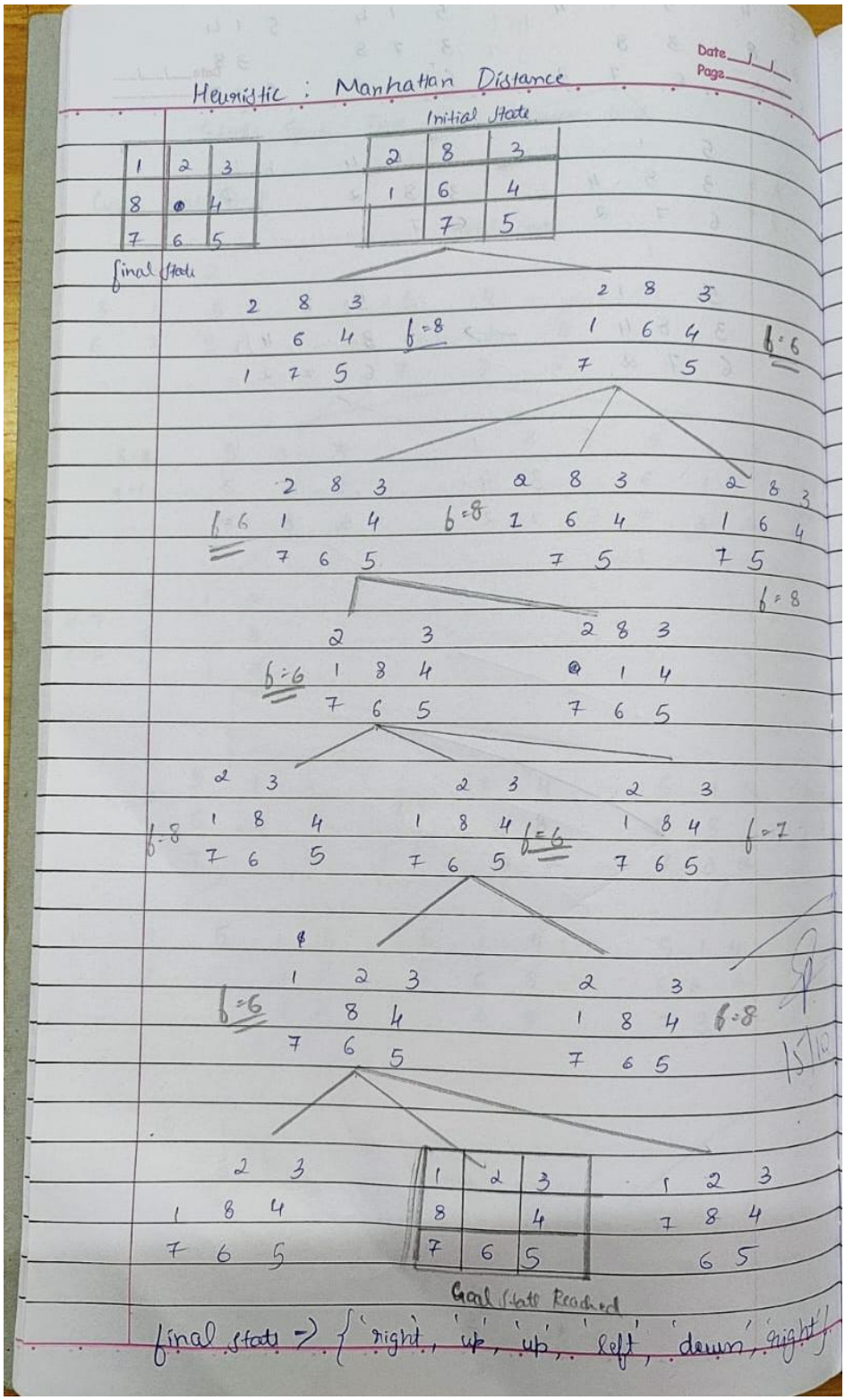
```
    return neighbors


def print_path(path):
    for state in path:
        for row in state:
            print(row)
        print()


def astar_manhattan(start, goal):
    open_list = [(start, 0, [])]  # (state, g(n), path)
    visited = set()

    while open_list:
        current, g, path = open_list.pop(0)
        path = path + [current]

        if current == goal:
            print("Solution Found (Manhattan Distance):")
            for state in path:
```

```
            distance = manhattan_distance(state, goal)
            print(f"State: {state} | Manhattan Distance: {distance}")
        print_path(path)
        return g  # Return the depth (number of moves)

    for neighbor in get_neighbors(current):
        neighbor_tuple = tuple(map(tuple, neighbor))
        if neighbor_tuple not in visited:
            visited.add(neighbor_tuple)
            h = manhattan_distance(neighbor, goal)
            open_list.append((neighbor, g + 1, path))

    open_list.sort(key=lambda x: x[1] + manhattan_distance(x[0], goal))

return -1  # No solution found

# Example usage
start = [[5, 4, 0], [6, 1, 8], [7, 3, 2]]
goal = [[0, 1, 2], [3, 4, 5], [6, 7, 8]]
astar_manhattan(start, goal)
```

OUTPUT :

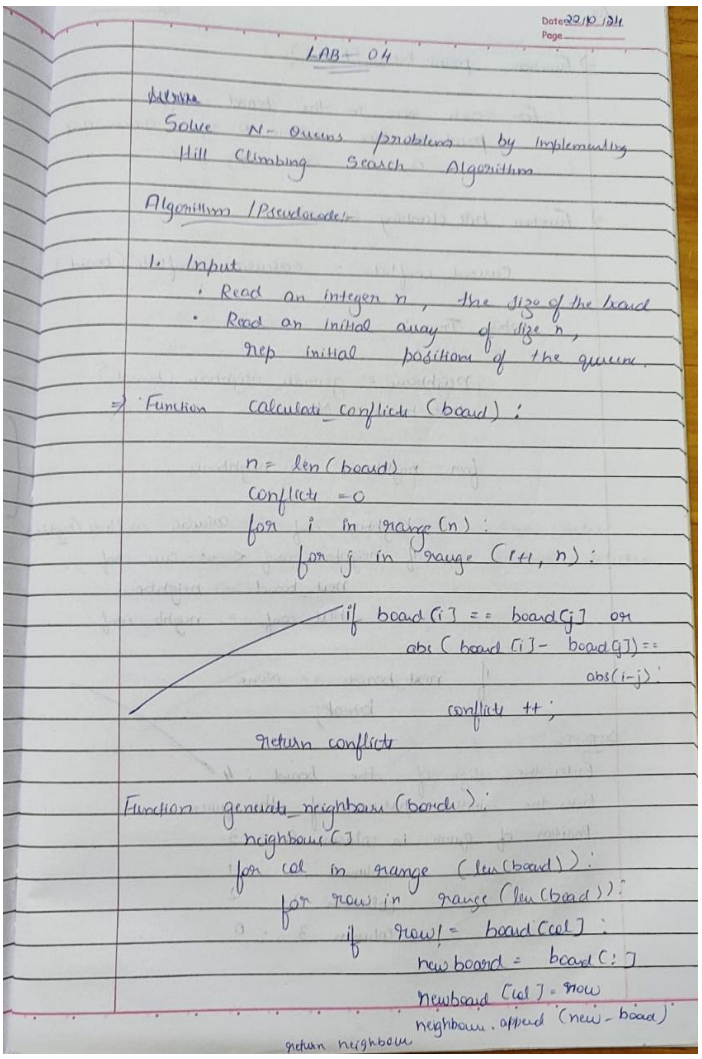
```
Solution Found (Manhattan Distance):
State: [[5, 4, 0], [6, 1, 8], [7, 3, 2]] | Manhattan Distance: 12
State: [[5, 0, 4], [6, 1, 8], [7, 3, 2]] | Manhattan Distance: 13
State: [[5, 1, 4], [6, 0, 8], [7, 3, 2]] | Manhattan Distance: 12
State: [[5, 1, 4], [6, 3, 8], [7, 0, 2]] | Manhattan Distance: 11
State: [[5, 1, 4], [6, 3, 8], [7, 2, 0]] | Manhattan Distance: 12
State: [[5, 1, 4], [6, 3, 0], [7, 2, 8]] | Manhattan Distance: 11
State: [[5, 1, 0], [6, 3, 4], [7, 2, 8]] | Manhattan Distance: 10
State: [[5, 0, 1], [6, 3, 4], [7, 2, 8]] | Manhattan Distance: 11
State: [[5, 3, 1], [6, 0, 4], [7, 2, 8]] | Manhattan Distance: 12
State: [[5, 3, 1], [6, 2, 4], [7, 0, 8]] | Manhattan Distance: 11
State: [[5, 3, 1], [6, 2, 4], [0, 7, 8]] | Manhattan Distance: 10
State: [[5, 3, 1], [0, 2, 4], [6, 7, 8]] | Manhattan Distance: 9
State: [[0, 3, 1], [5, 2, 4], [6, 7, 8]] | Manhattan Distance: 8
State: [[3, 0, 1], [5, 2, 4], [6, 7, 8]] | Manhattan Distance: 7
State: [[3, 1, 0], [5, 2, 4], [6, 7, 8]] | Manhattan Distance: 6
State: [[3, 1, 4], [5, 2, 0], [6, 7, 8]] | Manhattan Distance: 7
State: [[3, 1, 4], [5, 0, 2], [6, 7, 8]] | Manhattan Distance: 6
State: [[3, 1, 4], [0, 5, 2], [6, 7, 8]] | Manhattan Distance: 5
State: [[0, 1, 4], [3, 5, 2], [6, 7, 8]] | Manhattan Distance: 4
State: [[1, 0, 4], [3, 5, 2], [6, 7, 8]] | Manhattan Distance: 5
State: [[1, 4, 0], [3, 5, 2], [6, 7, 8]] | Manhattan Distance: 4
State: [[1, 4, 2], [3, 5, 0], [6, 7, 8]] | Manhattan Distance: 3
State: [[1, 4, 2], [3, 0, 5], [6, 7, 8]] | Manhattan Distance: 2
State: [[1, 0, 2], [3, 4, 5], [6, 7, 8]] | Manhattan Distance: 1
State: [[0, 1, 2], [3, 4, 5], [6, 7, 8]] | Manhattan Distance: 0
```

STATE SPACE TREE :



5.Implement N Queens using Hill climbing algorithm

ALGORITHM:



LAB - 04

Aim/Task
Solve N- Queens problems by implementing Hill Climbing Search Algorithm

Algorithm / Pseudocode:-

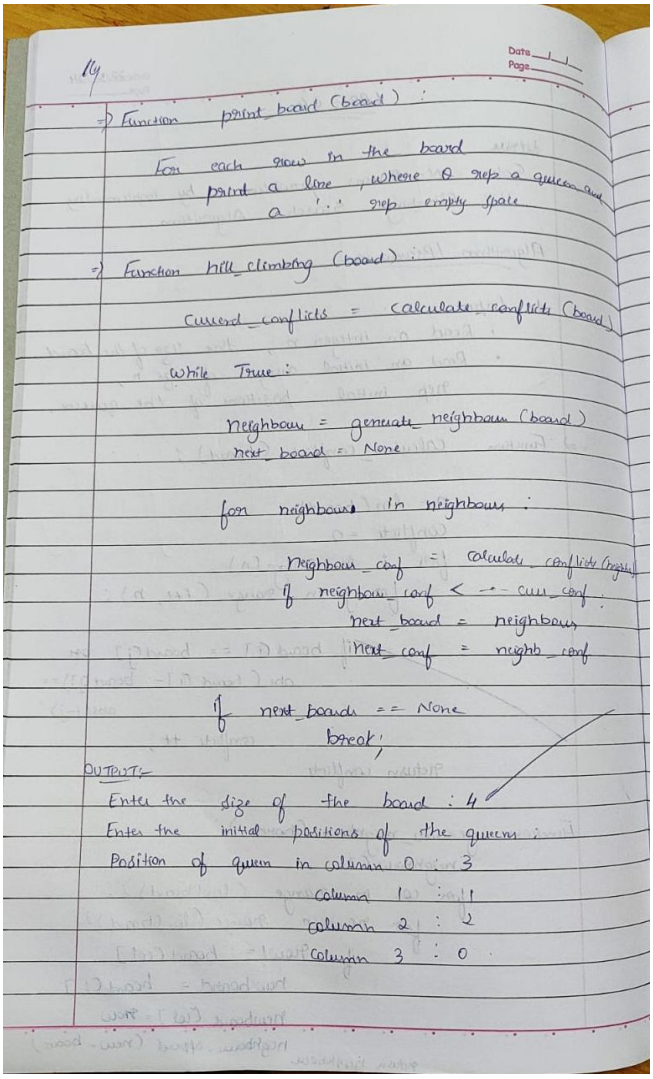
1. Input
   - Read an integer n, the size of the board
   - Read an initial array of size n, rep initial positions of the queens.

=> Function calculate_conflicts (board):

```
n = len(board)
conflicts = 0
for i in range(n):
    for j in range(i+1, n):
        if board[i] == board[j] or
           abs(board[i] - board[j]) ==
           abs(i-j):
            conflicts ++;
return conflicts
```

Function generate_neighbours (board):

```
neighbours []
for col in range (len(board)):
    for row in range (len(board)):
        if row != board[col]:
            new board = board[:]
            newboard[col] = row
            neighbours.append(new_board)
return neighbours
```



=> Function print_board (board):

For each row in the board
print a line, where Q rep a queen and a '.' rep empty space

=> Function hill_climbing (board):

```
current_conflicts = calculate_conflicts (board)
While True:
    neighbours = generate_neighbours (board)
    next_board = None
    for neighbour in neighbours:
        neighbour_conf = calculate_conflicts (neighbour)
        if neighbour_conf < current_conf:
            next_board = neighbour
            next_conf = neighb_conf
    if next_board == None
        break;
```

OUTPUT:-

```
Enter the size of the board : 4
Enter the initial positions of the queens :
Position of queen in column 0 : 3
Column 1 : 1
Column 2 : 2
Column 3 : 0
```

CODE:

```
# Function to calculate the heuristic
def calculate_heuristic(state):
    heuristic = 0
    n = len(state)
    for i in range(n):
```

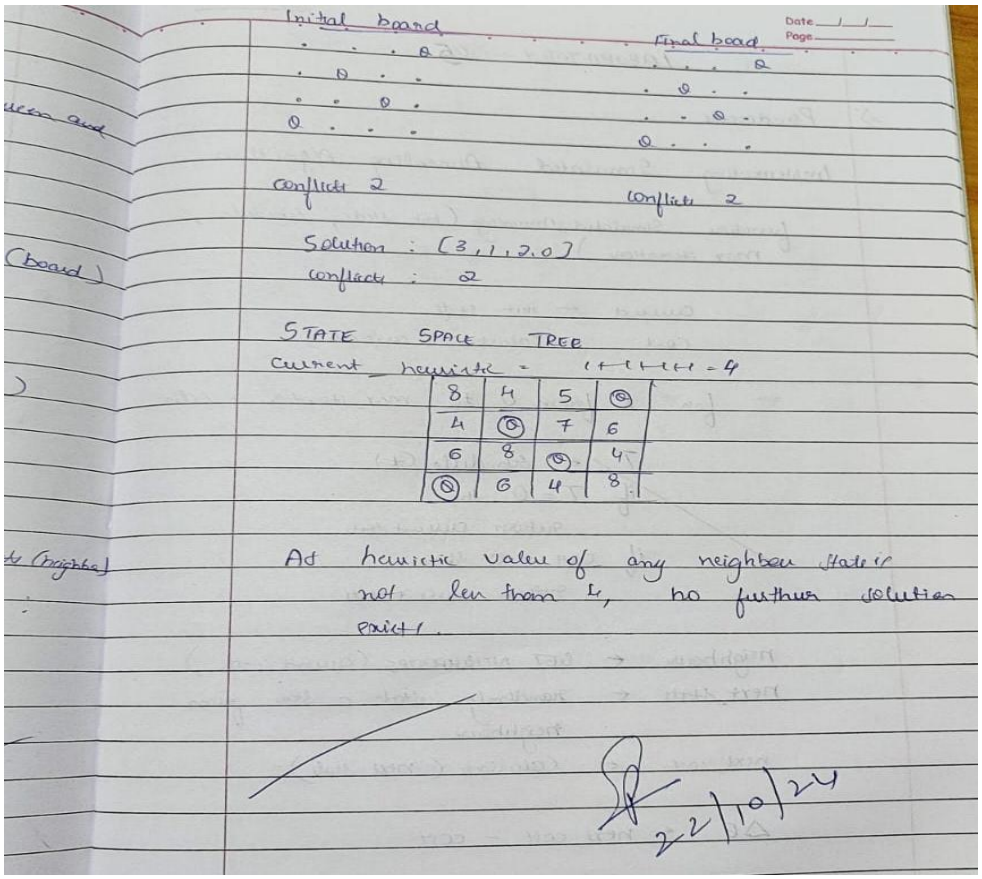
```
        for j in range(i + 1, n):
            if state[i] == state[j]:  # Same column
                heuristic += 1
            if abs(state[i] - state[j]) == abs(i - j):  # Same diagonal
                heuristic += 1
    return heuristic

# Function to generate neighboring states by swapping
def generate_neighbors(state):
    neighbors = []
    n = len(state)
    # Generate neighbors by swapping positions of two queens
    for i in range(n):
        for j in range(i + 1, n):
            new_state = state.copy()
            new_state[i], new_state[j] = new_state[j], new_state[i]  # Swap
            neighbors.append(new_state)
    return neighbors

# Function to print the board as a 2D array
def print_board(state):
    n = len(state)
    board = [['.'] * n for _ in range(n)]
    for row in range(n):
        board[row][state[row]] = 'Q'
    for row in board:
        print(' '.join(row))
    print()

# Hill Climbing algorithm for N-Queens with swapping
def hill_climbing_n_queens(initial_state):
    current_state = initial_state

    while True:
        current_heuristic = calculate_heuristic(current_state)
        print(f"Current State: {current_state}, Heuristic: {current_heuristic}")
        print_board(current_state)  # Print the current board

        if current_heuristic == 0:
```

```
            return current_state

        neighbors = generate_neighbors(current_state)
        best_neighbor = None
        best_heuristic = float('inf')

        for neighbor in neighbors:
            heuristic = calculate_heuristic(neighbor)
            if heuristic < best_heuristic:
                best_heuristic = heuristic
                best_neighbor = neighbor

        if best_heuristic >= current_heuristic:
            break  # Stop if no better neighbor found

        current_state = best_neighbor

    return None  # No solution found

# Main function to solve the N-Queens problem
def solve_n_queens():
    # Set the initial state for 4 queens
    initial_state = [3, 1, 2, 0]  # Fixed state
    solution = hill_climbing_n_queens(initial_state)

    if solution:
        print(f"Solution found for 4-Queens problem: {solution}")
        print_board(solution)  # Print the final board
    else:
        print("No solution found.")

# Run the solver
solve_n_queens()
```

OUTPUT :

```
Current State: [3, 1, 2, 0], Heuristic: 2
. . . Q
. Q . .
. . Q .
Q . . .

Current State: [1, 3, 2, 0], Heuristic: 1
. Q . .
. . . Q
. . Q .
Q . . .

Current State: [1, 3, 0, 2], Heuristic: 0
. Q . .
. . . Q
Q . . .
. . Q .

Solution found for 4-Queens problem: [1, 3, 0, 2]
. Q . .
. . . Q
Q . . .
. . Q .
```

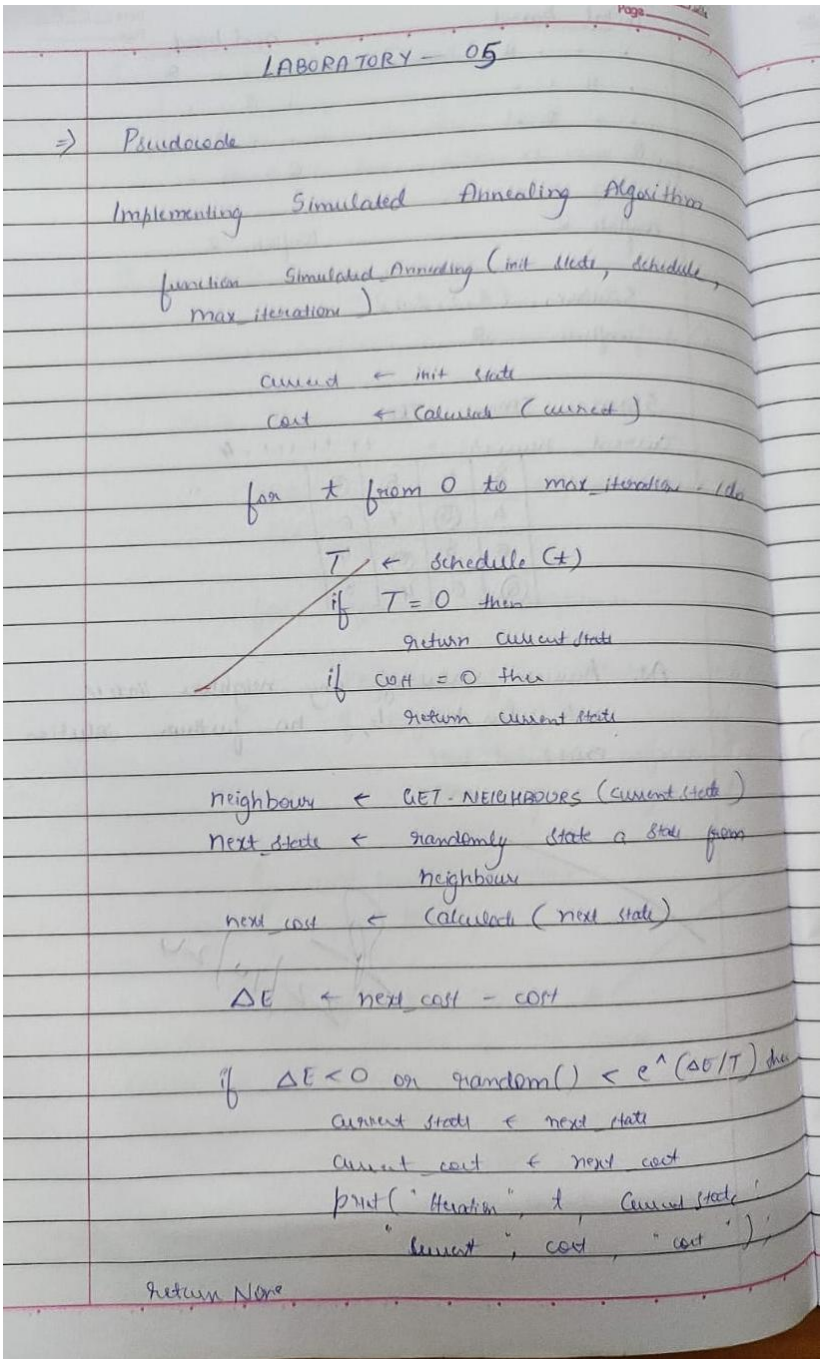
STATE-SPACE TREE:



WEEK-5

7. Simulated Annealing to solve N-Queens

ALGORITHM:



LABORATORY - 05

⇒ Pseudocode

Implementing Simulated Annealing Algorithm

function Simulated_Annealing (init state, schedule, max_iterations)

current ← init state
cost ← Calculate (current)

for t from 0 to max_iterations - 1 do

T ← schedule (t)
if T = 0 then
return current state
if cost = 0 then
return current state

neighbour ← GET-NEIGHBOURS (current state)
next_state ← randomly state a state from neighbour
next_cost ← Calculate (next state)

ΔE ← next_cost - cost

if ΔE < 0 or random() < e^(ΔE/T) then
current state ← next_state
current_cost ← next_cost
print ("Iteration", t, "current state", "current", cost, "cost");

return None

CODE:

```
import random
import math

def create_board(n, initial_config=None):
    """Create a board with the specified initial configuration."""
    if initial_config:
        return initial_config  # Use the provided initial configuration
    return [random.randint(0, n-1) for _ in range(n)]

def count_conflicts(board):
    """Count the number of pairs of queens that are attacking each other."""
    n = len(board)
    conflicts = 0
    for i in range(n):
        for j in range(i+1, n):
            if board[i] == board[j] or abs(board[i] - board[j]) == j - i:
                conflicts += 1
    return conflicts

def get_neighbors(board):
    """Generate neighbors by moving one queen at a time to a different row."""
    neighbors = []
    for i in range(len(board)):
        for row in range(len(board)):
            if row != board[i]:
                neighbor = board[:]
                neighbor[i] = row
                neighbors.append(neighbor)
    return neighbors

def print_board(board):
    """Print the board in a human-readable format."""
    n = len(board)
    for row in range(n):
        line = ['Q' if board[col] == row else '.' for col in range(n)]
        print(' '.join(line))
    print()
```

```
def simulated_annealing(n, initial_config, max_iterations=1000,
initial_temperature=100, cooling_rate=0.95):
    """Solve the N-Queens problem using simulated annealing."""
    current_board = create_board(n, initial_config)
    current_conflicts = count_conflicts(current_board)
    temperature = initial_temperature

    best_board = current_board[:]
    best_conflicts = current_conflicts

    # Print the initial configuration
    print("Initial configuration:")
    print_board(current_board)
    print(f"Initial number of conflicts: {current_conflicts}\n")

    for iteration in range(max_iterations):
        if current_conflicts == 0:
            # If the solution is found, print the final configuration and exit
            print(f"Solution found at iteration {iteration + 1}!")
            print("Final configuration (solution):")
            print_board(current_board)
            return current_board

        # Get neighbors of the current board
        neighbors = get_neighbors(current_board)
        next_board = random.choice(neighbors)
        next_conflicts = count_conflicts(next_board)

        # If the neighbor has fewer conflicts, accept it
        if next_conflicts < current_conflicts:
            current_board = next_board
            current_conflicts = next_conflicts
        else:
            # Accept the neighbor with a certain probability
            probability = math.exp((current_conflicts - next_conflicts) / temperature)
            if random.random() < probability:
                current_board = next_board
                current_conflicts = next_conflicts
```

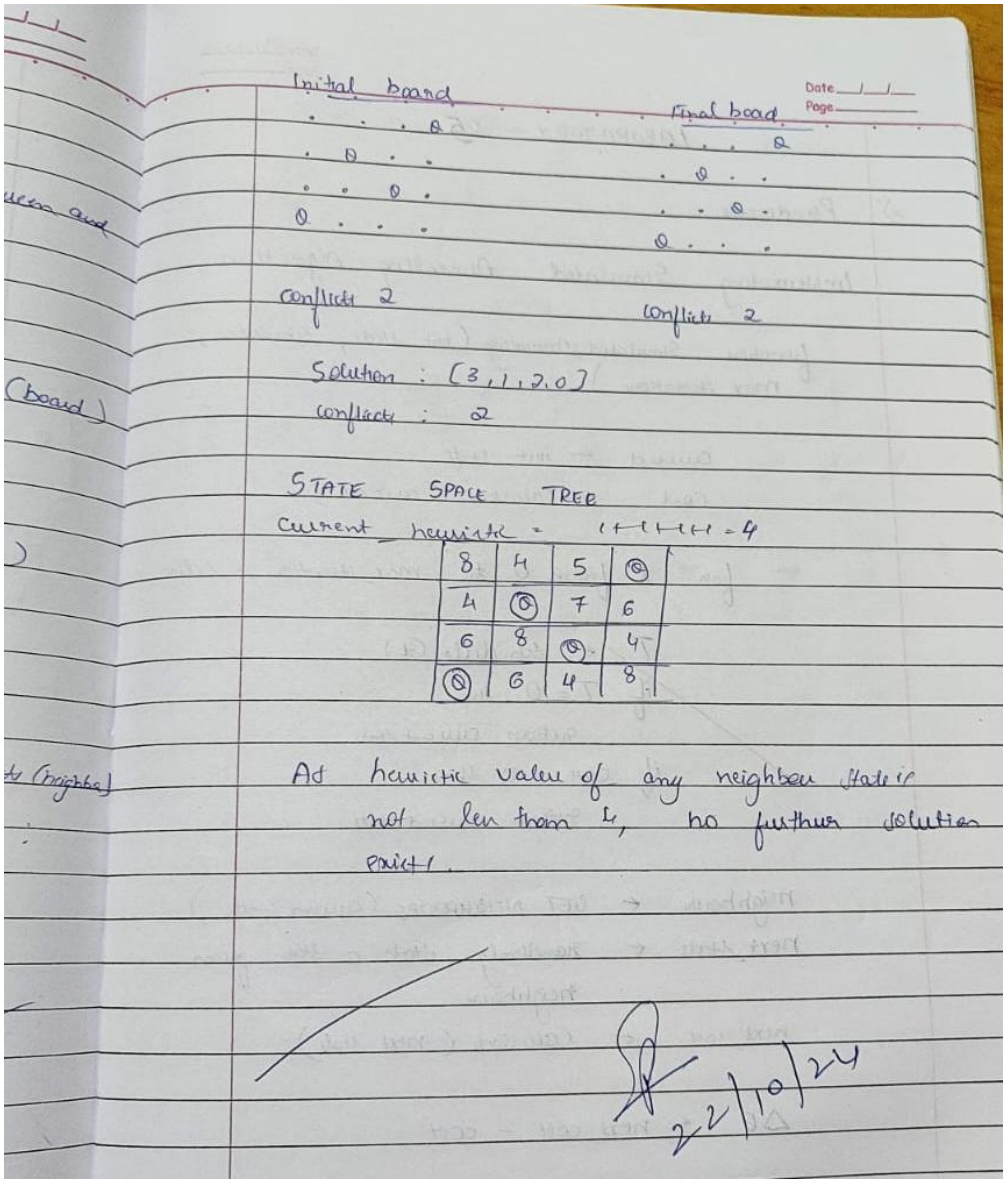
```
        # Update temperature
        temperature *= cooling_rate

    # If no solution is found, print the best state after max iterations
    print(f"No solution found within {max_iterations} iterations.")
    print("Best configuration found:")
    print_board(best_board)
    return best_board

# Set the number of queens (4 for this example) and initial configuration
n = 4
initial_config = [2, 1, 3, 0]  # The custom configuration
solution = simulated_annealing(n, initial_config)
```

OUTPUT:

Output

```
Initial configuration:
. . . Q
. Q . .
Q . . .
. . Q .

Initial number of conflicts: 1

Solution found at iteration 3!
Final configuration (solution):
. Q . .
. . . Q
Q . . .
. . Q .

=== Code Execution Successful ===
```

STATE SPACE TREE:



Initial board

. . . Q
. Q . .
. . Q .
Q . . .

Conflicts 2

Final board

. . . Q
. Q . .
. . Q .
Q . . .

Conflicts 2

Solution : [3, 1, 2, 0]
conflicts : 2

STATE SPACE TREE

Current heuristic = 1+1+1+1 = 4

8	4	5	(3)
4	(1)	7	6
6	8	(2)	4
(3)	6	4	8

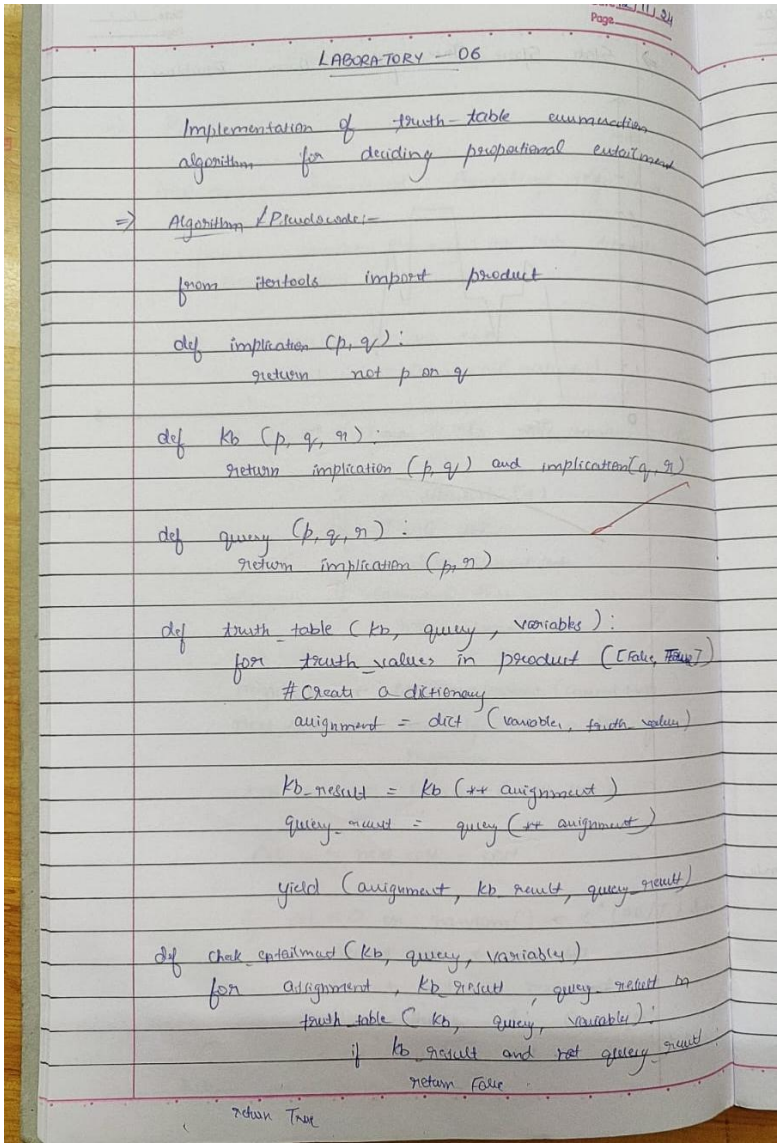
At heuristic value of any neighbour state is not less than 4, no further solution exists.

22/10/24

WEEK-6

8. Propositional Logic

ALGORITHM:



LABORATORY – 06

Implementation of truth-table enumeration algorithm for deciding propositional entailment

=> Algorithm / Pseudocode:-

```
from itertools import product

def implication (p, q):
    return not p or q

def kb (p, q, r):
    return implication (p, q) and implication (q, r)

def query (p, q, r):
    return implication (p, r)

def truth_table (kb, query, variables):
    for truth_values in product ([False, True])
        # Create a dictionary
        assignment = dict (variable, truth_values)

        kb_result = kb (** assignment)
        query_result = query (** assignment)

        yield (assignment, kb_result, query_result)

def check_entailment (kb, query, variables)
    for assignment, kb_result, query_result in
        truth_table (kb, query, variables):
            if kb_result and not query_result:
                return False
    return True
```

CODE:

```
combinations = [(True, True, True), (True, True, False), (True, False, True), (True, False, False),
                (False, True, True), (False, True, False), (False, False, True), (False, False, False)]
variable = {'p': 0, 'q': 1, 'r': 2}
kb = ''
q = ''
priority = {'~': 3, 'v': 1, '^': 2}

def input_rules():
    global kb, q
    kb = input("Enter rule: ")
    q = input("Enter the Query: ")

def entailment():
    global kb, q
    print('*' * 10 + "Truth Table Reference" + '*' * 10)
    print('kb', 'alpha')
    print('*' * 10)
    for comb in combinations:
        s = evaluatePostfix(toPostfix(kb), comb)
        f = evaluatePostfix(toPostfix(q), comb)
        print(s, f)
        print('-' * 10)
        if s and not f:
            return False
    return True

def isOperand(c):
    return c.isalpha() and c != 'v'

def isLeftParanthesis(c):
    return c == '('

def isRightParanthesis(c):
    return c == ')'

def isEmpty(stack):
    return len(stack) == 0

def peek(stack):
    return stack[-1]

def hasLessOrEqualPriority(c1, c2):
    try:
```

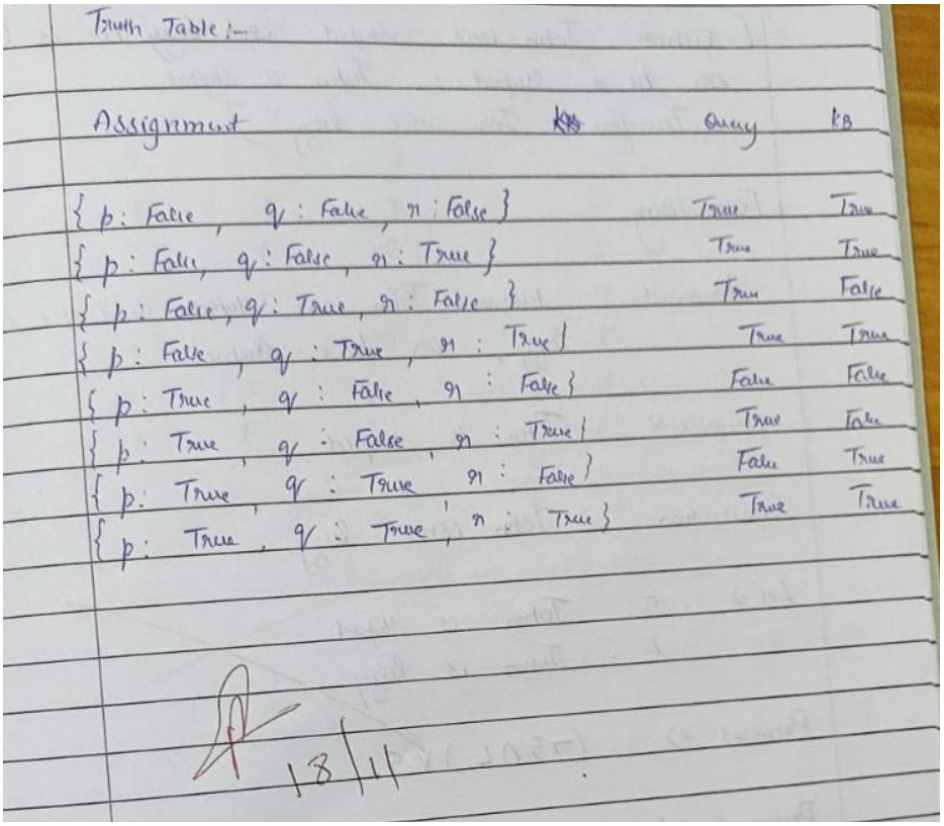
```
        return priority[c1] <= priority[c2]
    except KeyError:
        return False

def toPostfix(infix):
    stack = []
    postfix = ''
    for c in infix:
        if isOperand(c):
            postfix += c
        else:
            if isLeftParanthesis(c):
                stack.append(c)
            elif isRightParanthesis(c):
                operator = stack.pop()
                while not isLeftParanthesis(operator):
                    postfix += operator
                    operator = stack.pop()
            else:
                while (not isEmpty(stack)) and hasLessOrEqualPriority(c, peek(stack)):
                    postfix += stack.pop()
                stack.append(c)
    while (not isEmpty(stack)):
        postfix += stack.pop()
    return postfix

def evaluatePostfix(exp, comb):
    stack = []
    for i in exp:
        if isOperand(i):
            stack.append(comb[variable[i]])
        elif i == '~':
            val1 = stack.pop()
            stack.append(not val1)
        else:
            val1 = stack.pop()
            val2 = stack.pop()
            stack.append(_eval(i, val2, val1))
    return stack.pop()

def _eval(i, val1, val2):
    if i == '^':
        return val2 and val1
    return val2 or val1

input_rules()
```

```
ans = entailment()
if ans:
    print("The Knowledge Base entails query")
else:
    print("The Knowledge Base does not entail query")
```

OUTPUT:

```
Output

Initial configuration:
. . . Q
. Q . .
Q . . .
. . Q .

Initial number of conflicts: 1

Solution found at iteration 3!
Final configuration (solution):
. Q . .
. . . Q
Q . . .
. . Q .

=== Code Execution Successful ===
```

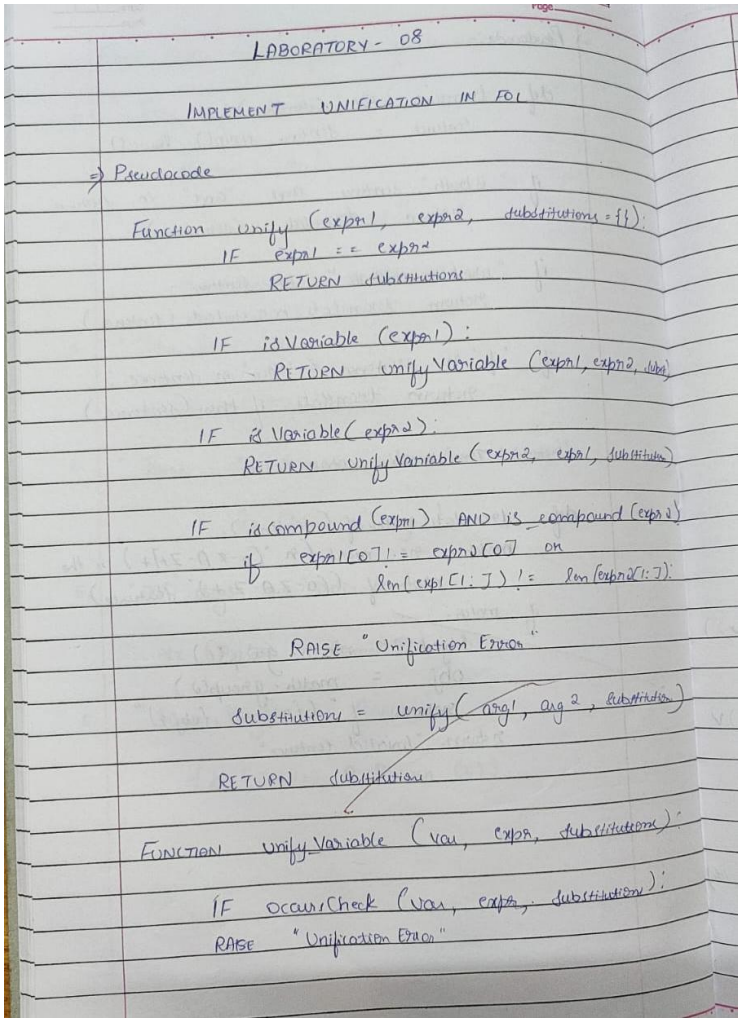
STATE SPACE TREE:



WEEK-7

Unification in First Order Logic

ALGORITHM:

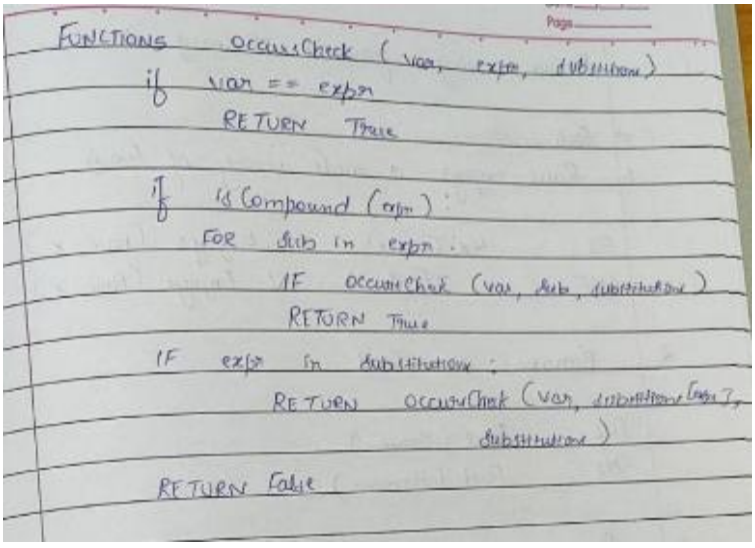


LABORATORY - 08

IMPLEMENT UNIFICATION IN FOL

=) Pseudocode

```
Function unify (expr1, expr2, substitutions={}):
    IF expr1 == expr2
        RETURN substitutions

    IF isVariable (expr1):
        RETURN unifyVariable (expr1, expr2, subs)

    IF is Variable(expr2):
        RETURN unifyVariable (expr2, expr1, substitution)

    IF isCompound (expr1) AND is_compound (expr2)
        if expr1[0] != expr2[0] or
            len(expr1[1:]) != len(expr2[1:]):

        RAISE "Unification Error"

    substitutions = unify( arg1, arg2, substitution)

    RETURN substitution

FUNCTION unifyVariable (var, expr, substitutions):

    IF occurCheck (var, expr, substitutions):
    RAISE "Unification Error"
```



```
FUNCTIONS occurCheck (var, expr, substitutions)
    if var == expr
        RETURN True

    if isCompound (expr)
        FOR sub in expr:
            IF occurCheck (var, sub, substitutions)
            RETURN True
    IF expr in substitutions:
        RETURN occurCheck (var, substitutions[expr],
                           substitutions)

    RETURN False
```

CODE:

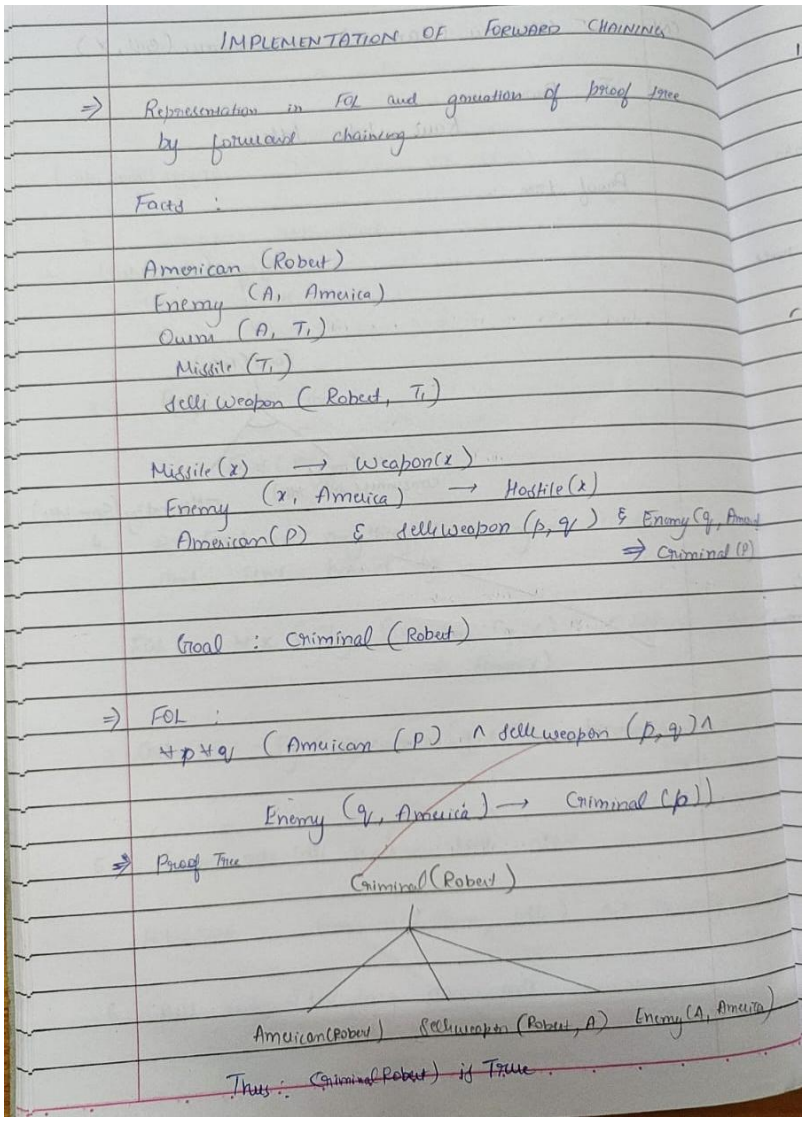
```
class UnificationError(Exception):
    pass

def unify(expr1, expr2, substitutions=None):
    if substitutions is None:
        substitutions = {}

    # If both expressions are identical, return current substitutions
    if expr1 == expr2:
        return substitutions

    # If the first expression is a variable
    if is_variable(expr1):
        return unify_variable(expr1, expr2, substitutions)

    # If the second expression is a variable
    if is_variable(expr2):
        return unify_variable(expr2, expr1, substitutions)

    # If both expressions are compound expressions
    if is_compound(expr1) and is_compound(expr2):
        if expr1[0] != expr2[0] or len(expr1[1:]) != len(expr2[1:]):
            raise UnificationError("Expressions do not match.")
        return unify_lists(expr1[1:], expr2[1:], unify(expr1[0], expr2[0],
substitutions))

    # If expressions are not compatible
    raise UnificationError(f"Cannot unify {expr1} and {expr2}.")

def unify_variable(var, expr, substitutions):
    if var in substitutions:
        return unify(substitutions[var], expr, substitutions)
    elif occurs_check(var, expr, substitutions):
        raise UnificationError(f"Occurs check failed: {var} in {expr}.")
    else:
        substitutions[var] = expr
        return substitutions
```

```
def unify_lists(list1, list2, substitutions):
    for expr1, expr2 in zip(list1, list2):
        substitutions = unify(expr1, expr2, substitutions)
    return substitutions

def is_variable(term):
    return isinstance(term, str) and term[0].islower()

def is_compound(term):
    return isinstance(term, (list, tuple)) and len(term) > 0

def occurs_check(var, expr, substitutions):
    if var == expr:
        return True
    elif is_compound(expr):
        return any(occurs_check(var, sub, substitutions) for sub in expr)
    elif expr in substitutions:
        return occurs_check(var, substitutions[expr], substitutions)
    return False

# Example usage
try:
    expr1 = ("f", "x", ("g", "y"))
    expr2 = ("f", "a", ("g", "b"))
    result = unify(expr1, expr2)
    print("Unified substitutions:", result)
except UnificationError as e:
    print("Unification failed:", e)
```

WEEK-8

10.Forward Chaining

ALGORITHM:



CODE:

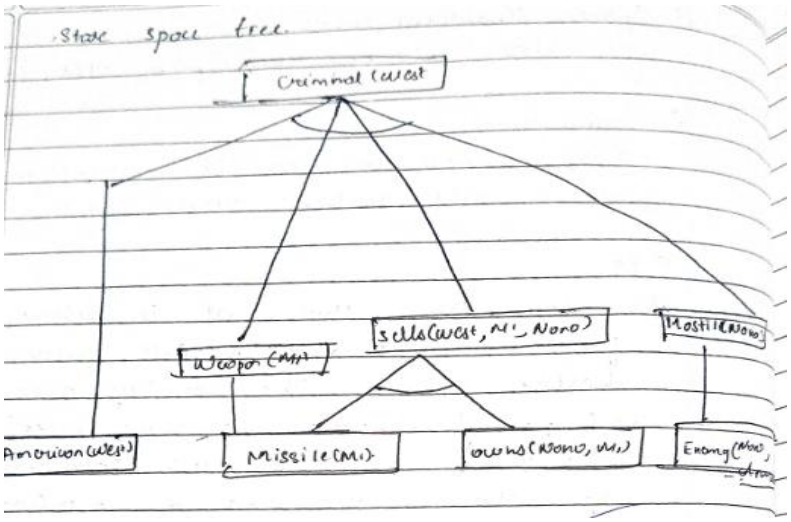
```
# Define initial facts and rules
facts = {"InAmerica(West)", "SoldWeapons(West, Nono)", "Enemy(Nono,
America)"}
rules = [
    {
        "conditions": ["InAmerica(x)", "SoldWeapons(x, y)", "Enemy(y, America)"],
        "conclusion": "Criminal(x)",
    },
    {
        "conditions": ["Enemy(y, America)"],
        "conclusion": "Dangerous(y)",
    },
]

# Forward chaining function
def forward_chaining(facts, rules):
    derived_facts = set(facts)  # Initialize derived facts
    while True:
        new_fact_found = False

        for rule in rules:
            # Substitute variables and check if conditions are met
            for fact in derived_facts:
                if "x" in rule["conditions"][0]:
                    # Substitute variables (x, y) with specific instances
                    for condition in rule["conditions"]:
                        if "x" in condition or "y" in condition:
                            x = "West"  # Hardcoded substitution for simplicity
                            y = "Nono"
                            conditions = [
                                cond.replace("x", x).replace("y", y)
                                for cond in rule["conditions"]
                            ]
                            conclusion = (
                                rule["conclusion"].replace("x", x).replace("y", y)
                            )
```

```
                    # Check if all conditions are satisfied
                    if all(cond in derived_facts for cond in conditions) and
conclusion not in derived_facts:
                        derived_facts.add(conclusion)
                        print(f"New fact derived: {conclusion}")
                        new_fact_found = True

        # Exit loop if no new fact is found
        if not new_fact_found:
            break

    return derived_facts

# Run forward chaining
final_facts = forward_chaining(facts, rules)
print("\nFinal derived facts:")
for fact in final_facts:
    print(fact)
```

OUTPUT :

```
Output

Final derived facts:
Enemy(Nono, America)
InAmerica(West)
SoldWeapons(West, Nono)

=== Code Execution Successful ===
```

STATE SPACE TREE:



WEEK-8

Resolution of First Order Logic statements

ALGORITHM:

CODE:

```
def negate(literal):
    """Return the negation of a literal."""
    if isinstance(literal, tuple) and literal[0] == "not":
        return literal[1]
    else:
        return ("not", literal)

def resolve(clause1, clause2):
    """Return the resolvent of two clauses."""
    resolvents = set()
    for literal1 in clause1:
        for literal2 in clause2:
            if literal1 == negate(literal2):
                resolvent = (clause1 - {literal1}) | (clause2 - {literal2})
                print(f"    Resolving literal: {literal1} with {literal2}")
                print(f"    Resulting Resolvent: {resolvent}")
                resolvents.add(frozenset(resolvent))
    return resolvents

def resolution_algorithm(KB, query):
    """Perform the resolution algorithm to check if the query can be proven."""
    print("\n--- Step-by-Step Resolution Process ---")
    negated_query = negate(query)
    KB.append(frozenset([negated_query]))
    print(f"Negated Query Added to KB: {negated_query}")

    clauses = set(KB)

    step = 1
    while True:
        new_clauses = set()
```

```
        print(f"\nStep {step}: Resolving Clauses")
        for c1 in clauses:
            for c2 in clauses:
                if c1 != c2:
                    print(f"  Resolving clauses: {c1} and {c2}")
                    resolvent = resolve(c1, c2)
                    for res in resolvent:
                        if frozenset([]) in resolvent:
                            print("\nEmpty clause derived! The query is provable.")
                            return True
                        new_clauses.add(res)

        if new_clauses.issubset(clauses):
            print("\nNo new clauses can be derived. The query is not provable.")
            return False

        clauses.update(new_clauses)
        step += 1

KB = [
    frozenset([("not", "food(x)"), ("likes", "John", "x")]),
    frozenset([("food", "Apple")]),
    frozenset([("food", "vegetables")]),
    frozenset([("not", "eats(y, z)"), ("killed", "y"), ("food", "z")]),
    frozenset([("eats", "Anil", "Peanuts")]),
    frozenset([("alive", "Anil")]),
    frozenset([("not", "eats(Anil, w)"), ("eats", "Harry", "w")]),
    frozenset([("killed", "g"), ("alive", "g")]),
    frozenset([("not", "alive(k)"), ("not", "killed(k)")]),
    frozenset([("likes", "John", "Peanuts")])
]


query = ("likes", "John", "Peanuts")

result = resolution_algorithm(KB, query)
if result:
    print("\nQuery is provable.")
else:
    print("\nQuery is not provable.")
```

OUTPUT :

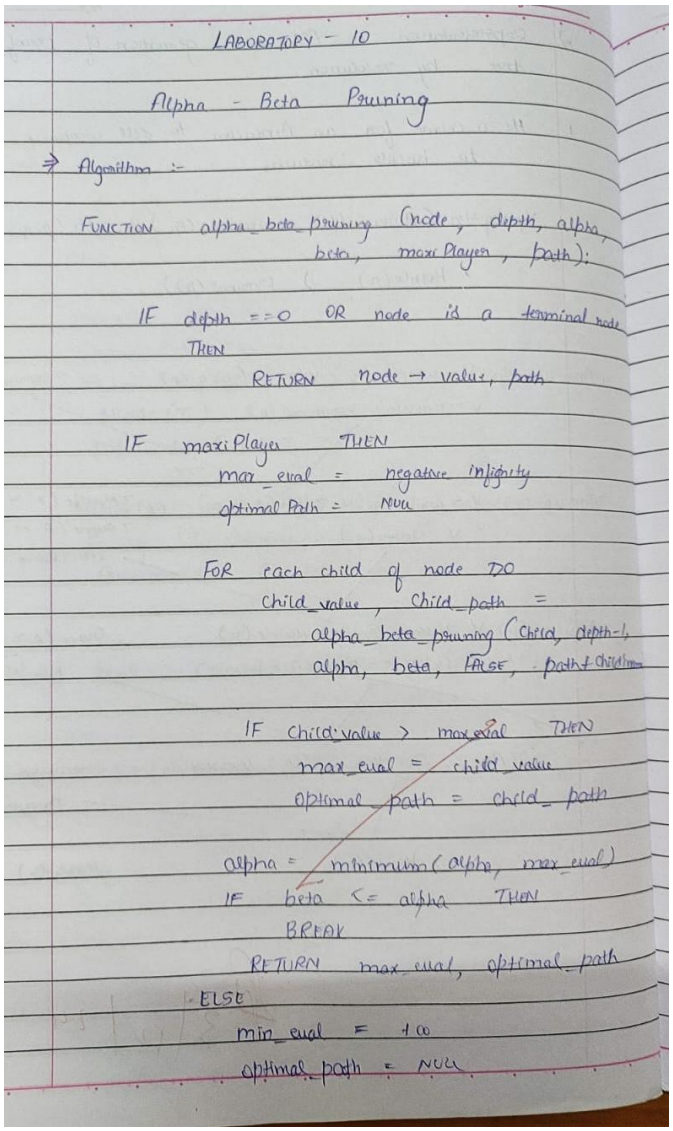
```
  Resolving clauses: frozenset({('food', 'Apple')}) and frozenset({('alive', 'g'), ('killed', 'g')})
  Resolving clauses: frozenset({('food', 'Apple')}) and frozenset({('eats', 'Harry', 'w'), ('not', 'eats(Anil, w)')})
  Resolving clauses: frozenset({('food', 'Apple')}) and frozenset({('not', 'killed(k)'), ('not', 'alive(k)')})
  Resolving clauses: frozenset({('food', 'Apple')}) and frozenset({('eats', 'Anil', 'Peanuts')})
  Resolving clauses: frozenset({('food', 'Apple')}) and frozenset({('not', ('likes', 'John', 'Peanuts'))})
  Resolving clauses: frozenset({('food', 'Apple')}) and frozenset({('food', 'vegetables')})
  Resolving clauses: frozenset({('not', 'food(x)'), ('likes', 'John', 'x')}) and frozenset({('food', 'Apple')})
  Resolving clauses: frozenset({('not', 'food(x)'), ('likes', 'John', 'x')}) and frozenset({('killed', 'y'), ('not', 'eats(y, z)'), ('food', 'z')})
  Resolving clauses: frozenset({('not', 'food(x)'), ('likes', 'John', 'x')}) and frozenset({('likes', 'John', 'Peanuts')})
  Resolving clauses: frozenset({('not', 'food(x)'), ('likes', 'John', 'x')}) and frozenset({('alive', 'Anil')})
  Resolving clauses: frozenset({('not', 'food(x)'), ('likes', 'John', 'x')}) and frozenset({('alive', 'g'), ('killed', 'g')})
  Resolving clauses: frozenset({('not', 'food(x)'), ('likes', 'John', 'x')}) and frozenset({('eats', 'Harry', 'w'), ('not', 'eats(Anil, w)')})
  Resolving clauses: frozenset({('not', 'food(x)'), ('likes', 'John', 'x')}) and frozenset({('not', 'killed(k)'), ('not', 'alive(k)')})
  Resolving clauses: frozenset({('not', 'food(x)'), ('likes', 'John', 'x')}) and frozenset({('eats', 'Anil', 'Peanuts')})
  Resolving clauses: frozenset({('not', 'food(x)'), ('likes', 'John', 'x')}) and frozenset({('not', ('likes', 'John', 'Peanuts'))})
  Resolving clauses: frozenset({('not', 'food(x)'), ('likes', 'John', 'x')}) and frozenset({('food', 'vegetables')})
  Resolving clauses: frozenset({('killed', 'y'), ('not', 'eats(y, z)'), ('food', 'z')}) and frozenset({('food', 'Apple')})
  Resolving clauses: frozenset({('killed', 'y'), ('not', 'eats(y, z)'), ('food', 'z')}) and frozenset({('not', 'food(x)'), ('likes', 'John', 'x')})
  Resolving clauses: frozenset({('killed', 'y'), ('not', 'eats(y, z)'), ('food', 'z')}) and frozenset({('likes', 'John', 'Peanuts')})
  Resolving clauses: frozenset({('killed', 'y'), ('not', 'eats(y, z)'), ('food', 'z')}) and frozenset({('alive', 'Anil')})
  Resolving clauses: frozenset({('killed', 'y'), ('not', 'eats(y, z)'), ('food', 'z')}) and frozenset({('alive', 'g'), ('killed', 'g')})
  Resolving clauses: frozenset({('killed', 'y'), ('not', 'eats(y, z)'), ('food', 'z')}) and frozenset({('eats', 'Harry', 'w'), ('not', 'eats(Anil, w)')})
  Resolving clauses: frozenset({('killed', 'y'), ('not', 'eats(y, z)'), ('food', 'z')}) and frozenset({('not', 'killed(k)'), ('not', 'alive(k)')})
  Resolving clauses: frozenset({('killed', 'y'), ('not', 'eats(y, z)'), ('food', 'z')}) and frozenset({('eats', 'Anil', 'Peanuts')})
  Resolving clauses: frozenset({('killed', 'y'), ('not', 'eats(y, z)'), ('food', 'z')}) and frozenset({('not', ('likes', 'John', 'Peanuts'))})
  Resolving clauses: frozenset({('killed', 'y'), ('not', 'eats(y, z)'), ('food', 'z')}) and frozenset({('food', 'vegetables')})
  Resolving clauses: frozenset({('likes', 'John', 'Peanuts')}) and frozenset({('food', 'Apple')})
  Resolving clauses: frozenset({('likes', 'John', 'Peanuts')}) and frozenset({('not', 'food(x)'), ('likes', 'John', 'x')})
  Resolving clauses: frozenset({('likes', 'John', 'Peanuts')}) and frozenset({('killed', 'y'), ('not', 'eats(y, z)'), ('food', 'z')})
  Resolving clauses: frozenset({('likes', 'John', 'Peanuts')}) and frozenset({('alive', 'Anil')})
  Resolving clauses: frozenset({('likes', 'John', 'Peanuts')}) and frozenset({('alive', 'g'), ('killed', 'g')})
  Resolving clauses: frozenset({('likes', 'John', 'Peanuts')}) and frozenset({('eats', 'Harry', 'w'), ('not', 'eats(Anil, w)')})
  Resolving clauses: frozenset({('likes', 'John', 'Peanuts')}) and frozenset({('not', 'killed(k)'), ('not', 'alive(k)')})
  Resolving clauses: frozenset({('likes', 'John', 'Peanuts')}) and frozenset({('eats', 'Anil', 'Peanuts')})
  Resolving clauses: frozenset({('likes', 'John', 'Peanuts')}) and frozenset({('not', ('likes', 'John', 'Peanuts'))})
    Resolving literal: ('likes', 'John', 'Peanuts') with ('not', ('likes', 'John', 'Peanuts'))
    Resulting Resolvent: frozenset()

Empty clause derived! The query is provable.
```

TRACING :

WEEK-9

Alpha-Beta Pruning

ALGORITHM :



LABORATORY - 10

Alpha - Beta Pruning

⇒ Algorithm :-

FUNCTION alpha_beta_pruning (node, depth, alpha, beta, maxiPlayer, path):

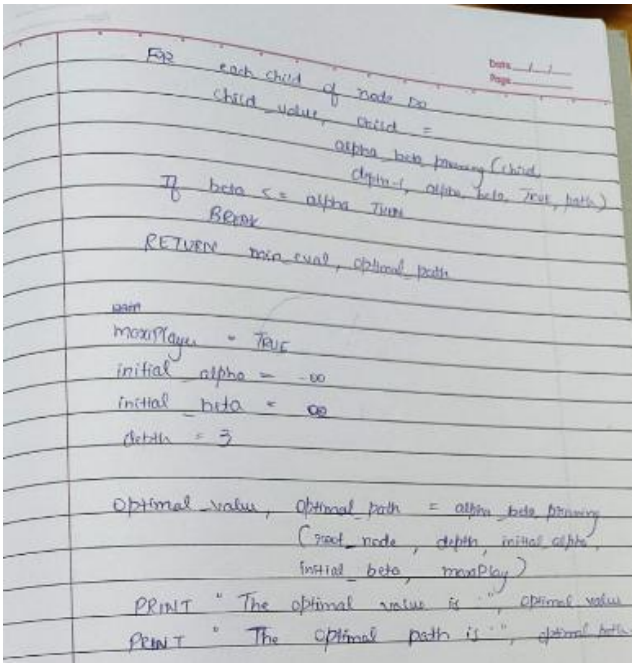
IF depth == 0 OR node is a terminal node THEN
    RETURN node → value, path

IF maxiPlayer THEN
    max_eval = negative infinity
    optimal Path = NULL

    FOR each child of node DO
        child_value, child_path = alpha_beta_pruning (child, depth-1, alpha, beta, FALSE, path+child)

        IF child_value > max_eval THEN
            max_eval = child_value
            optimal_path = child_path

        alpha = minimum (alpha, max_eval)
        IF beta <= alpha THEN
            BREAK
    RETURN max_eval, optimal_path
ELSE
    min_eval = +∞
    optimal_path = NULL



FOR each child of node DO
    child_value, child = alpha_beta_pruning (child, depth-1, alpha, beta, True, path)
    If beta <= alpha THEN
        BREAK
RETURN min_eval, optimal_path

main
maxiPlayer = TRUE
initial_alpha = -∞
initial_beta = ∞
depth = 3

Optimal_value, optimal_path = alpha_beta_pruning (root_node, depth, initial_alpha, initial_beta, maxiPlay)
PRINT "The optimal value is :", optimal_value
PRINT "The optimal path is :", optimal_path

CODE:

```
def alpha_beta_search(state):

    def max_value(state, alpha, beta, path):
```

```
    print(f"MAX: Visiting state {state}, alpha={alpha}, beta={beta}")
    if terminal_test(state):
        print(f"MAX: Terminal state {state} has utility {utility(state)}")
        return utility(state), path
    v = float('-inf')
    best_path = []
    for action in actions(state):
        result_state = result(state, action)
        value, new_path = min_value(result_state, alpha, beta, path + [action])
        print(f"MAX: From state {state}, action {action} → value={value}")
        if value > v:
            v = value
            best_path = new_path
        if v >= beta:
            print(f"MAX: Pruning at state {state} with value={v} ≥ beta={beta}")
            return v, best_path
        alpha = max(alpha, v)
    print(f"MAX: Returning value={v} for state {state}")
    return v, best_path

def min_value(state, alpha, beta, path):
    print(f"MIN: Visiting state {state}, alpha={alpha}, beta={beta}")
    if terminal_test(state):
        print(f"MIN: Terminal state {state} has utility {utility(state)}")
        return utility(state), path
```

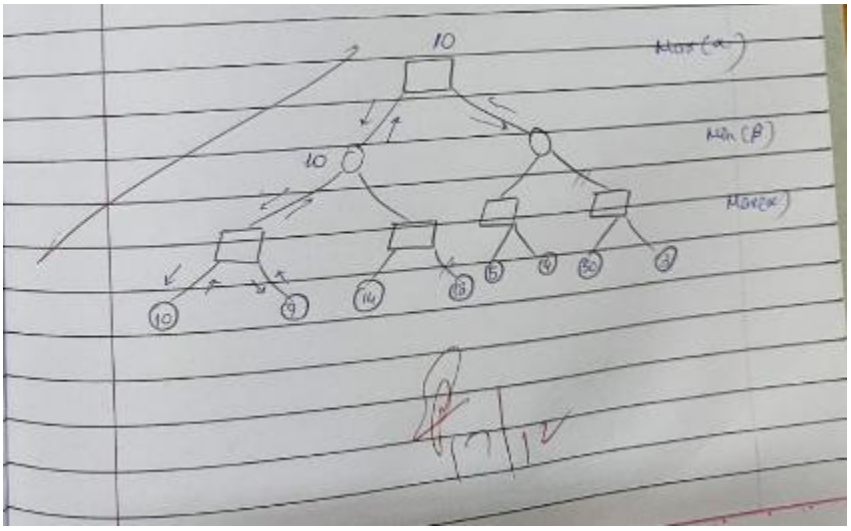
```
        v = float('inf')
        best_path = []
        for action in actions(state):
            result_state = result(state, action)
            value, new_path = max_value(result_state, alpha, beta, path + [action])
            print(f"MIN: From state {state}, action {action} → value={value}")
            if value < v:
                v = value
                best_path = new_path
            if v <= alpha:
                print(f"MIN: Pruning at state {state} with value={v} ≤ alpha={alpha}")
                return v, best_path
            beta = min(beta, v)
        print(f"MIN: Returning value={v} for state {state}")
        return v, best_path

    print("Starting Alpha-Beta Search...\n")
    final_value, final_path = max_value(state, float('-inf'), float('inf'), [state])
    return final_value, final_path


def terminal_test(state):
    return state in terminal_states


def utility(state):
```

```
    return terminal_states[state]


def actions(state):
    return game_tree.get(state, [])
```

OUTPUT :

```
Starting Alpha-Beta Search...

MAX: Visiting state A, alpha=-inf, beta=inf
MIN: Visiting state B, alpha=-inf, beta=inf
MAX: Visiting state D, alpha=-inf, beta=inf
MIN: Visiting state H, alpha=-inf, beta=inf
MIN: Terminal state H has utility 10
MAX: From state D, action H → value=10
MIN: Visiting state I, alpha=10, beta=inf
MIN: Terminal state I has utility 9
MAX: From state D, action I → value=9
MAX: Returning value=10 for state D
MIN: From state B, action D → value=10
MAX: Visiting state E, alpha=-inf, beta=10
MIN: Visiting state J, alpha=-inf, beta=10
MIN: Terminal state J has utility 14
MAX: From state E, action J → value=14
MAX: Pruning at state E with value=14 ≥ beta=10
MIN: From state B, action E → value=14
MIN: Returning value=10 for state B
MAX: From state A, action B → value=10
MIN: Visiting state C, alpha=10, beta=inf
MAX: Visiting state F, alpha=10, beta=inf
MIN: Visiting state L, alpha=10, beta=inf
MIN: Terminal state L has utility 5
MAX: From state F, action L → value=5
MIN: Visiting state M, alpha=10, beta=inf
MIN: Terminal state M has utility 4
MAX: From state F, action M → value=4
MAX: Returning value=5 for state F
MIN: From state C, action F → value=5
MIN: Pruning at state C with value=5 ≤ alpha=10
MAX: From state A, action C → value=5
MAX: Returning value=10 for state A

--- Final Results ---
The final utility value is: 10
The final path taken is: ['A', 'B', 'D', 'H']
```

STATE SPACE TREE :


10
Max(α)
10
Min(β)
Max(α)
10
9
14
18
15
4
80
2